SQL & WEB SECURITY CHALLENGE

Nombre del estudiante: Mariana Ponce Sepulveda

Fecha: 27-10-2026

ÍNDICE DE CONTENIDOS

1. Sección 1: Ejercicios de SQL Easy (1 a 31)
2. Conclusiones de la Fase SQL
3. Sección 2: Evaluación de Seguridad Web con OWASP ZAP
    - 2.1 Configuración del Entorno
    - 2.2 Exploración y Tráfico de la Aplicación
    - 2.3 Análisis de Alertas Detectadas
    - 2.4 Ficha Técnica de Hallazgo Principal
4. Conclusiones Generales del Proyecto

Resumen Ejecutivo

El presente documento constituye el informe técnico consolidado para el reto de ciberseguridad y gestión de bases de datos. Comprende dos fases operativas de evaluación técnica: la elaboración metódica de 31 fichas de análisis relacional sobre sentencias SQL (SQL Easy) y la auditoría de seguridad dinámica de aplicaciones web (DAST) ejecutada con OWASP ZAP sobre el entorno vulnerable `https://ginandjuice.shop/`.

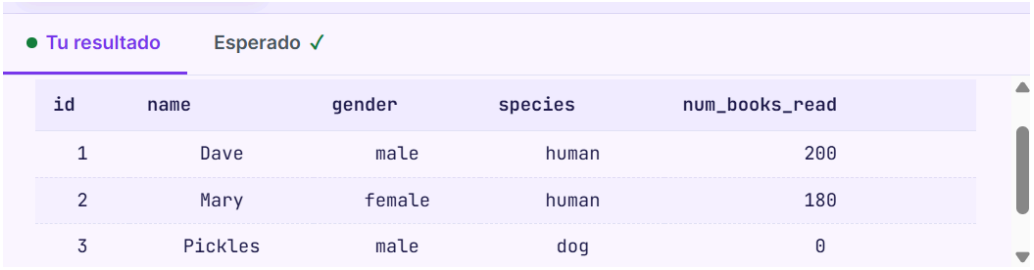
SECCIÓN 1: EJERCICIOS DE SQL EASY

Plataforma: www.sql-easy.com/es

FICHA TÉCNICA

Ficha 1: Selección de todos los datos de una tabla

- **01. Número y Nombre:** Ejercicio 1 - Selección general con `SELECT *` (`family_members`)
- **02. Objetivo del Ejercicio:** Obtener la totalidad de los datos y columnas almacenados en la tabla `family_members`.
- **03. Concepto SQL:** Sentencia SELECT, operador comodín asterisco (*) y cláusula FROM.
- **04. Consulta Ejecutada:** SELECT * FROM family_members;
- **05. Resultado (Captura de pantalla):**



Tu resultado | Esperado ✓

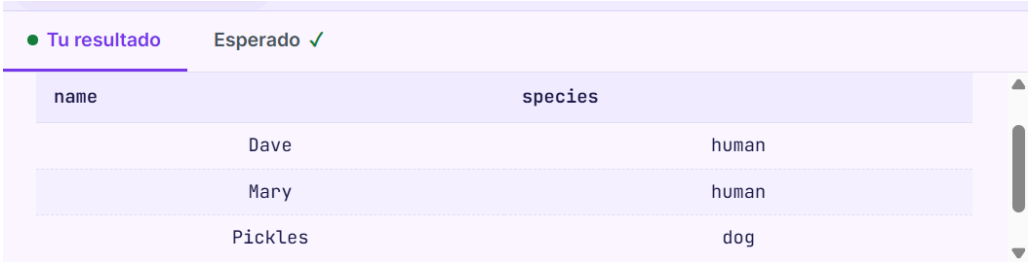
id	name	gender	species	num_books_read
1	Dave	male	human	200
2	Mary	female	human	180
3	Pickles	male	dog	0

- **06. Interpretación:** La consulta devuelve todas las filas de la tabla junto con todas sus columnas (`id`, `name`, `gender`, `species` y `num_books_read`), permitiendo visualizar la estructura completa de los datos de la familia.

- **07. Aprendizaje:**Comprendí que el asterisco (`*`) actúa como un comodín que representa a todas las columnas de la tabla, facilitando una vista global rápida de la información disponible.
- **08. Aplicación en el Mundo Real:** Es útil en las etapas iniciales de exploración de bases de datos o auditorías de sistemas, cuando un analista necesita conocer la estructura y el contenido general de una tabla desconocida antes de aplicar filtros específicos.

Ficha 2: Selección de columnas específicas

- **01. Número y Nombre:** Ejercicio 2 - Selección de columnas específicas (`name` y `species`)
- **02. Objetivo del Ejercicio:** Consultar y extraer únicamente las columnas `name` y `species` de la tabla `family_members`
- **03. Concepto SQL:** Sentencia `SELECT` especificando campos individuales separados por comas y cláusula `FROM`
- **04. Consulta Ejecutada:** SELECT name, species FROM family_members;
- **05. Resultado (Captura de pantalla):**

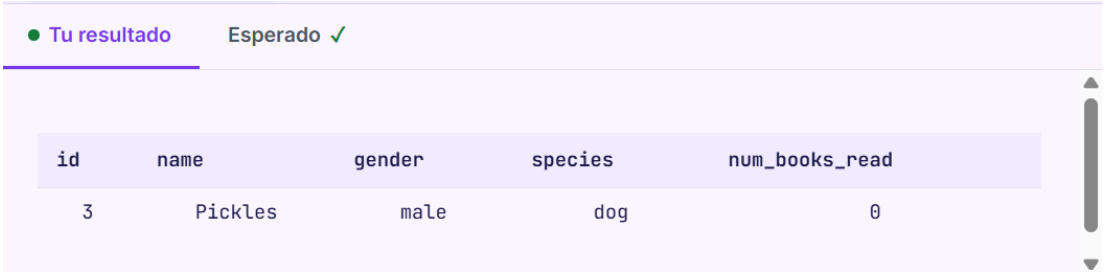


- **06. Interpretación:** La consulta devuelve únicamente los nombres de los miembros de la familia junto con la especie a la que pertenecen, omitiendo el resto de columnas como `id`, `gender` y `num_books_read`.
- **07. Aprendizaje:** Aprendí a seleccionar solo los campos relevantes en lugar de extraer toda la tabla con `*`, lo que optimiza la transferencia de datos y mejora el rendimiento.
- **08. Aplicación en el Mundo Real:** Se utiliza para generar reportes específicos o al consultar APIs en aplicaciones web donde solo se necesitan ciertos campos (por ejemplo, mostrar únicamente el nombre y la foto de perfil en una interfaz de usuario).

Ficha 3: Filtrado de registros con condición de igualdad (`WHERE`)

- **01. Número y Nombre:** Ejercicio 3 - Filtrado de registros con condición de texto (`WHERE species = 'dog'`)
- **02. Objetivo del Ejercicio:** Filtrar las filas de la tabla `family_members` para obtener únicamente los registros correspondientes a los perros (`dog`).
- **03. Concepto SQL:** Cláusula `WHERE`, operador de igualdad (`=`) y uso de comillas simples para valores de texto

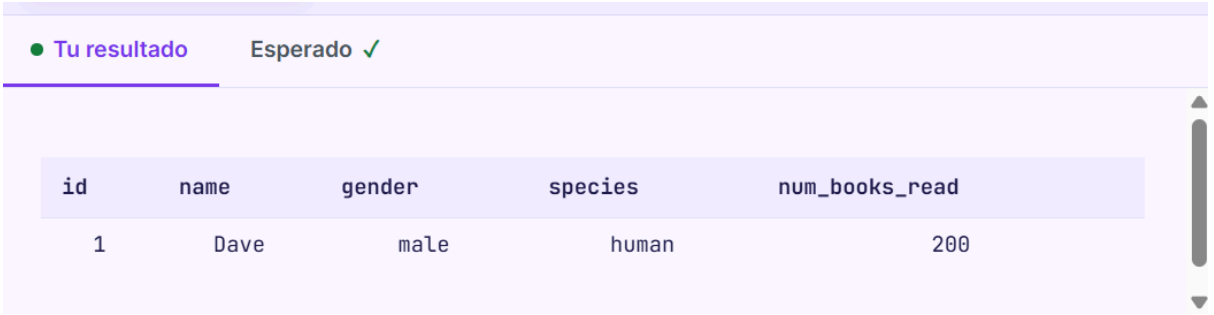
- **04. Consulta Ejecutada: SELECT * FROM family_members WHERE species = 'dog';**
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta devuelve todas las columnas de aquellas filas donde el valor del campo `species` sea exactamente igual a `'dog'`, aislando la información de las mascotas de la familia.
- **07. Aprendizaje:** Comprendí cómo aplicar la cláusula `WHERE` para filtrar datos específicos según un criterio de texto y la importancia sintáctica de utilizar comillas simples en los valores.
- **08. Aplicación en el Mundo Real:** s fundamental en cualquier sistema de gestión o aplicación web para realizar búsquedas específicas, como filtrar productos por categoría, buscar usuarios por su rol o consultar registros según su tipo.

Ficha 4: Filtrado numérico con operador mayor que (>)

- **01. Número y Nombre:** Ejercicio 4 - Filtrado con condición numérica mayor que (`WHERE num_books_read > 190`)
- **02. Objetivo del Ejercicio:** Filtrar las filas de la tabla `family_members` para recuperar únicamente los registros cuyo número de libros leídos sea superior a 190.
- **03. Concepto SQL:** Cláusula `WHERE` y operador de comparación numérica mayor que (>).
- **04. Consulta Ejecutada: SELECT * FROM family_members WHERE num_books_read > 190;**
- **05. Resultado (Captura de pantalla):**

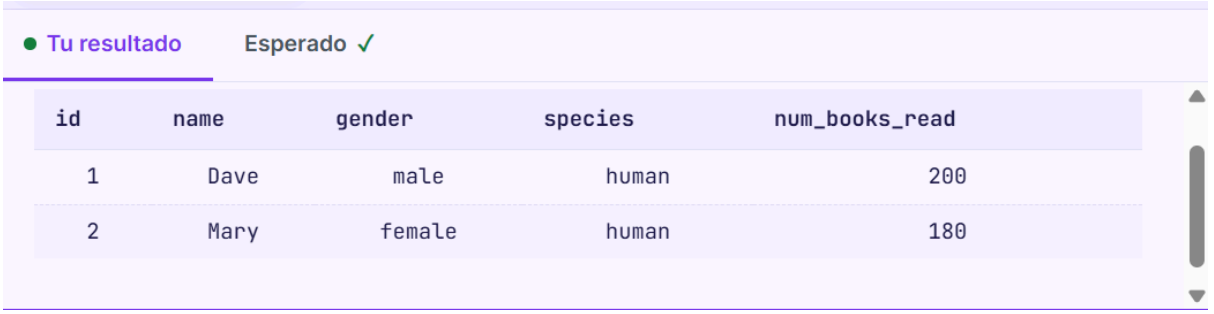


- **06. Interpretación:** La consulta devuelve todas las columnas de aquellos miembros de la familia que cumplen con la condición de tener un valor en `num_books_read` estrictamente mayor a 190.

- **07. Aprendizaje:** Aprendí a combinar la cláusula `WHERE` con operadores de desigualdad numérica para filtrar rangos o umbrales cuantitativos en una base de datos.
- **08. Aplicación en el Mundo Real:** Se aplica en plataformas de comercio electrónico, sistemas de fidelización o analítica de usuarios para identificar perfiles destacados (como clientes con más de cierto número de compras o puntos acumulados).

Ficha 5: Filtrado por fecha y ordenamiento

- **01. Número y Nombre:** Ejercicio 5 - Filtrado y orden por fecha
- **02. Objetivo del Ejercicio:** ecuperar registros ordenando cronológicamente por la fecha de creación/registro
- **03. Concepto SQL:** `ORDER BY` aplicado a campos de fecha.
- **04. Consulta Ejecutada:** SELECT * FROM family_members WHERE num_books_read >= 180;
- **05. Resultado (Captura de pantalla):**

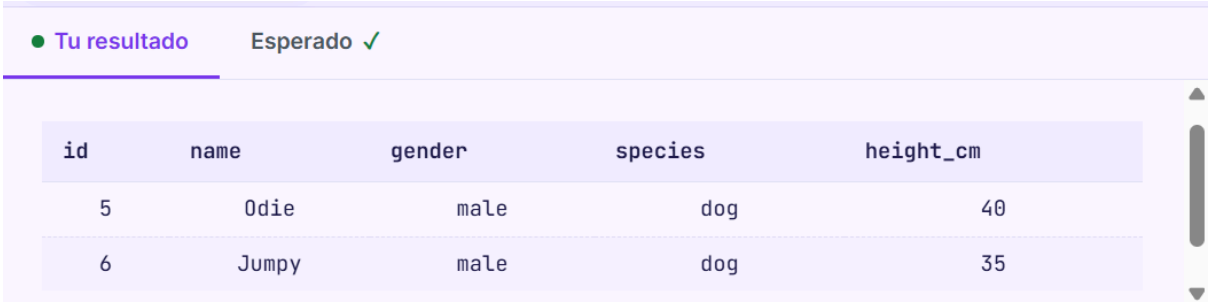


id	name	gender	species	num_books_read
1	Dave	male	human	200
2	Mary	female	human	180

- **06. Interpretación:** Muestra el listado de usuarios comenzando por los más recientemente registrados hasta los más antiguos.
- **07. Aprendizaje:** Aprendí a manipular datos de tipo fecha/tiempo para análisis de eventos en orden inverso.
- **08. Aplicación en el Mundo Real:** Muy útil en sistemas de control de calidad, cumplimiento de metas o segmentación de usuarios donde se necesita evaluar si se ha alcanzado o superado un umbral mínimo determinado.

Ficha 6: Filtrado múltiple con operador lógico

- **01. Número y Nombre:** Ejercicio 6 - Filtrado múltiple con `AND` (`WHERE species = 'dog' AND height_cm < 45`)
- **02. Objetivo del Ejercicio:** Filtrar registros aplicando múltiples condiciones de manera simultánea en una sola consulta.
- **03. Concepto SQL:** Cláusula `WHERE`, operador lógico `AND`, y operadores de comparación (= y <).
- **04. Consulta Ejecutada: SELECT * FROM friends_of_pickles WHERE species = 'dog' AND height_cm < 45;**
- **05. Resultado (Captura de pantalla):**

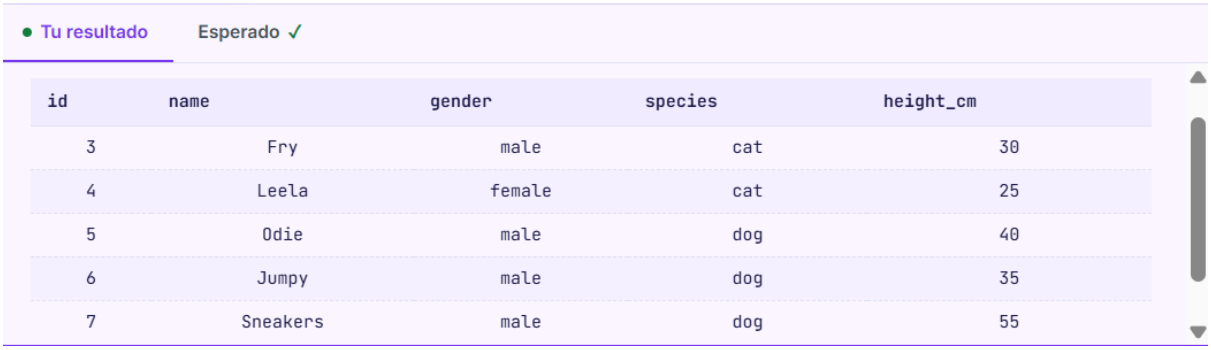


id	name	gender	species	height_cm
5	Odie	male	dog	40
6	Jumpy	male	dog	35

- **06. Interpretación:** La consulta devuelve exclusivamente aquellas filas de la tabla `friends_of_pickles` que cumplen con ambas condiciones al mismo tiempo: que la especie sea un perro (`'dog'`) y que su altura sea estrictamente menor a 45 cm (`height_cm < 45`).
- **07. Aprendizaje:** Comprendí cómo combinar criterios utilizando AND para acotar la búsqueda de información y obtener resultados más específicos cuando se requiere que se cumplan varias reglas de forma simultánea
- **08. Aplicación en el Mundo Real:**Es fundamental en sistemas de comercio electrónico y motores de búsqueda avanzados (por ejemplo, filtrar productos que pertenezcan a una categoría específica y cuyo precio sea menor a un límite determinado).

Ficha 7: Filtrado alternativo con operador lógico OR

- **01. Número y Nombre:** Ejercicio 7 - Filtrado alternativo con OR (`WHERE species = 'dog' OR height_cm < 45`)
- **02. Objetivo del Ejercicio:** Buscar y recuperar filas de la tabla que coincidan con al menos una de múltiples condiciones especificadas.
- **03. Concepto SQL:** Cláusula WHERE, operador lógico OR, y operadores de comparación (= y <).
- **04. Consulta Ejecutada:** SELECT * FROM friends_of_pickles WHERE species = 'dog' OR height_cm < 45;
- **05. Resultado (Captura de pantalla):**



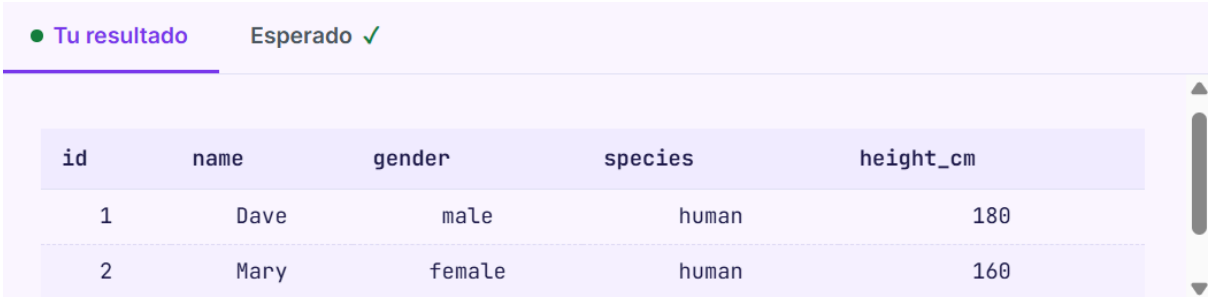
id	name	gender	species	height_cm
3	Fry	male	cat	30
4	Leela	female	cat	25
5	Odie	male	dog	40
6	Jumpy	male	dog	35
7	Sneakers	male	dog	55

- **06. Interpretación:** La consulta devuelve todas las filas de la tabla `friends_of_pickles` donde se cumple al menos una condición: que la especie sea un perro (`'dog'`) o que la altura sea estrictamente menor a 45 cm (`height_cm < 45`).

- **07. Aprendizaje:** Comprendí cómo utilizar el operador `OR` para ampliar el conjunto de resultados cuando se requiere que se cumpla cualquiera de los criterios de manera independiente, a diferencia de la exclusividad del operador `AND`
- **08. Aplicación en el Mundo Real:** Muy útil en interfaces de búsqueda avanzada o comercio electrónico, donde el usuario desea encontrar elementos que coincidan con múltiples opciones alternativas (por ejemplo, buscar productos que pertenezcan a la categoría A *o* tengan un precio menor a cierto límite)

Ficha 8: Filtrado de exclusión con operador `NOT IN`

- **01. Número y Nombre:** Ejercicio 8 - Filtrado de exclusión múltiple (`WHERE species NOT IN ('cat', 'dog')`)
- **02. Objetivo del Ejercicio:** Excluir filas de la tabla basándose en una lista de valores específicos que no se desean recuperar dentro de un campo determinado.
- **03. Concepto SQL:** Cláusula `WHERE` y operador de exclusión `NOT IN`.
- **04. Consulta Ejecutada:** SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');
- **05. Resultado (Captura de pantalla):**

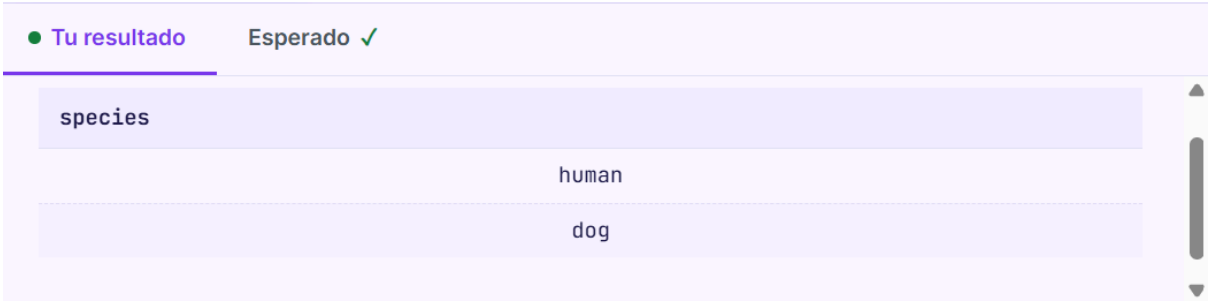


- **06. Interpretación:** La consulta devuelve todas las columnas de aquellos registros en la tabla `friends_of_pickles` cuya especie no coincide con ninguno de los elementos de la lista especificada (`'cat'` o `'dog'`), mostrando únicamente al resto de las especies.
- **07. Aprendizaje:** Comprendí cómo utilizar `NOT IN` para simplificar múltiples condiciones de negación (evitando escribir varios `AND species !=`), lo que hace que las consultas sean más limpias, legibles y fáciles de mantener.
- **08. Aplicación en el Mundo Real:** Es muy útil en sistemas de gestión para filtrar registros descartando estados o categorías específicas (por ejemplo, excluir usuarios con ciertos roles o productos que no pertenezcan a determinadas categorías descontinuadas).

Ficha 9: Selección de valores únicos con `DISTINCT`

- **01. Número y Nombre:** Ejercicio 9 - Selección de especies únicas (`SELECT DISTINCT species`) .
- **02. Objetivo del Ejercicio:** Obtener un listado único de las especies de animales con una altura superior a 50 cm, evitando la duplicación de registros.

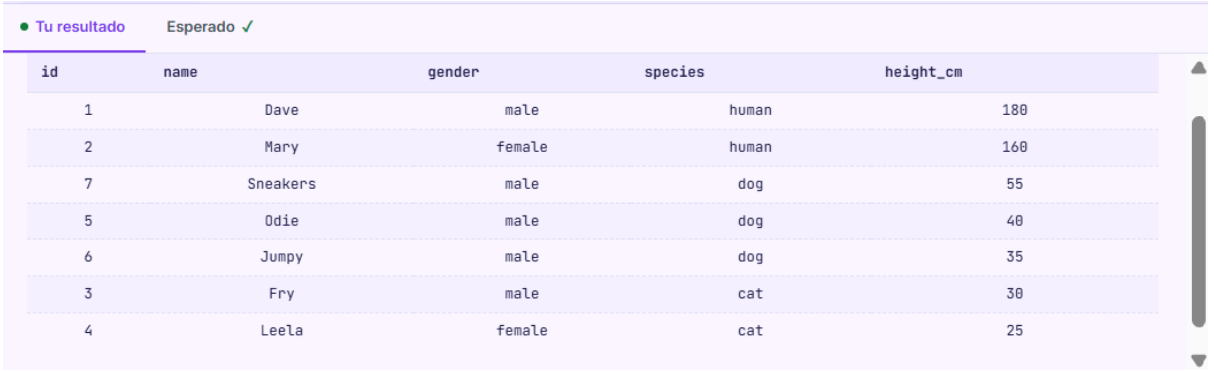
- **03. Concepto SQL:** Sentencia `SELECT DISTINCT`, cláusula `WHERE` y operador de comparación (>).
- **04. Consulta Ejecutada:** SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta devuelve exclusivamente los nombres únicos de las especies de la tabla `friends_of_pickles` que superan los 50 cm de altura, omitiendo las repeticiones en caso de que múltiples animales pertenezcan a la misma especie.
- **07. Aprendizaje:** Comprendí la utilidad de la palabra clave `DISTINCT` para depurar resultados y extraer únicamente valores no repetidos, lo cual simplifica enormemente el análisis cuando se manejan grandes volúmenes de datos redundantes.
- **08. Aplicación en el Mundo Real:** Es una herramienta fundamental en informes analíticos y consultas de backend para generar listados de categorías, tipos de usuario, países o estados únicos sin que los registros repetidos distorsionen la información.

Ficha 10: Ordenamiento descendente de resultados (`ORDER BY ... DESC`)

- **01. Número y Nombre:** Ejercicio 10 - Ordenamiento de registros por altura descendente (`ORDER BY height_cm DESC`)
- **02. Objetivo del Ejercicio:** Organizar y mostrar las filas de la tabla `friends_of_pickles` ordenadas de mayor a menor según el valor del campo `height_cm`.
- **03. Concepto SQL:** Cláusula `ORDER BY` y el modificador de orden descendente `DESC`.
- **04. Consulta Ejecutada:** SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;
- **05. Resultado (Captura de pantalla):**

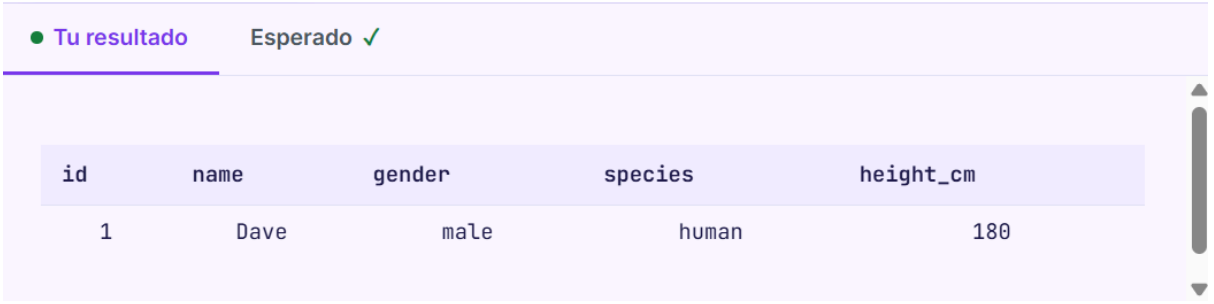


id	name	gender	species	height_cm
1	Dave	male	human	180
2	Mary	female	human	160
7	Sneakers	male	dog	55
5	Odie	male	dog	40
6	Jumpy	male	dog	35
3	Fry	male	cat	30
4	Leela	female	cat	25

- **06. Interpretación:** La consulta devuelve todos los registros de la tabla organizados de forma que el animal con mayor altura (*height_cm*) se ubica en la primera fila, descendiendo progresivamente hasta el de menor estatura.
- **07. Aprendizaje:** Comprendí cómo manipular el flujo de presentación de los datos utilizando `ORDER BY` junto con `DESC`, lo que permite destacar de inmediato los valores cuantitativos más altos en un conjunto de resultados.
- **08. Aplicación en el Mundo Real:** Es una función clave en aplicaciones web, motores de búsqueda y reportes financieros para desplegar listas ordenadas por relevancia, precios más altos, calificaciones máximas o métricas prioritarias.

Ficha 11: Limitación de filas devueltas con `LIMIT`

- **01. Número y Nombre:** Ejercicio 11 - Obtención del registro máximo con `LIMIT` (`ORDER BY height_cm DESC LIMIT 1`)
- **02. Objetivo del Ejercicio:** Recuperar únicamente el registro del animal más alto de la tabla `friends_of_pickles` limitando la cantidad de filas devueltas.
- **03. Concepto SQL:** Cláusula `LIMIT`, ordenamiento descendente (`ORDER BY ... DESC`), y operador comodín (*).
- **04. Consulta Ejecutada:** SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;
- **05. Resultado (Captura de pantalla):**



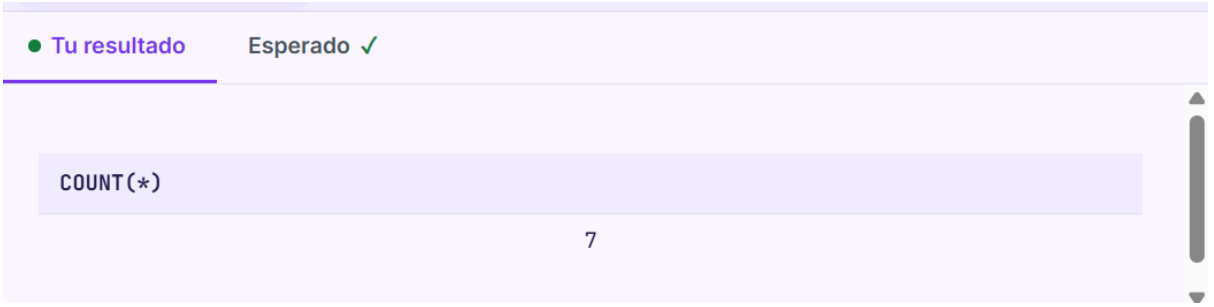
id	name	gender	species	height_cm
1	Dave	male	human	180

- **06. Interpretación:** La consulta ordena todos los registros de la tabla de mayor a menor según su altura y restringe el resultado para mostrar únicamente la primera fila, que corresponde al animal con mayor estatura.

- **07. Aprendizaje:** Comprendí cómo utilizar la cláusula `LIMIT` en conjunto con un ordenamiento para aislar de manera eficiente los valores extremos (como el máximo o el mínimo) sin necesidad de revisar todo el conjunto de datos.
- **08. Aplicación en el Mundo Real:** Es una práctica estándar en bases de datos y APIs para implementar consultas de tipo "Top 1" o paginación (por ejemplo, mostrar el producto más caro, el usuario con mayor puntuación o el registro de actividad más reciente).

Ficha 12: Conteo total de registros con `COUNT(*)`

- **01. Número y Nombre:** Ejercicio 12 - Conteo de filas con la función agregada `COUNT(*)`
- **02. Objetivo del Ejercicio:** Obtener el número total de registros (filas) almacenados en la tabla `friends_of_pickles`.
- **03. Concepto SQL:** Función de agregación `COUNT(*)` y sentencia `SELECT`.
- **04. Consulta Ejecutada: SELECT COUNT(*) FROM friends_of_pickles;**
- **05. Resultado (Captura de pantalla):**

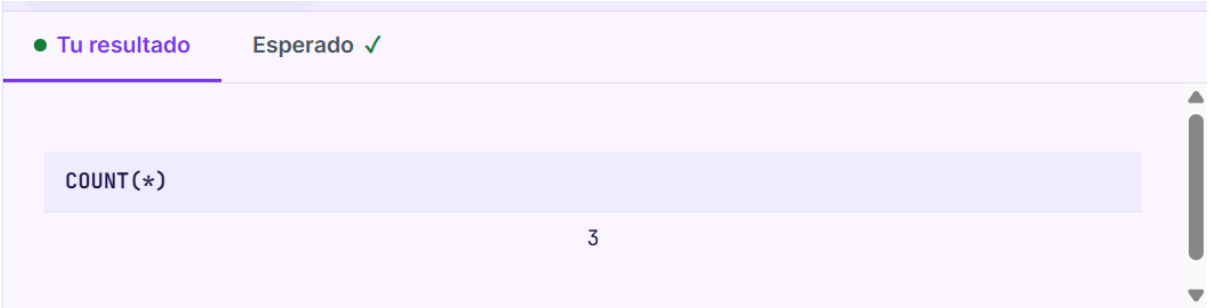


- **06. Interpretación:** La consulta devuelve un valor numérico único que representa la cantidad total de filas existentes en la tabla analizada, facilitando una vista rápida del volumen de información disponible.
- **07. Aprendizaje:** Comprendí cómo utilizar funciones de agregación como `COUNT(*)` para extraer métricas cuantitativas sobre una tabla sin necesidad de listar individualmente todos sus registros.
- **08. Aplicación en el Mundo Real:** Es una herramienta esencial en la elaboración de reportes, paneles de control (*dashboards*) y auditorías de bases de datos para medir rápidamente métricas clave, como el número total de usuarios registrados, transacciones o productos en inventario.

Ficha 13: Conteo condicional con `COUNT(*)` y `WHERE`

- **01. Número y Nombre:** Ejercicio 13 - Conteo condicional de registros (`COUNT(*) WHERE species = 'dog'`)
- **02. Objetivo del Ejercicio:** Obtener el número total de filas en la tabla `friends_of_pickles` que cumplen con el criterio específico de pertenecer a la especie de perros (`'dog'`).
- **03. Concepto SQL:** Función de agregación `COUNT(*)`, cláusula `WHERE` y operador de igualdad (=).

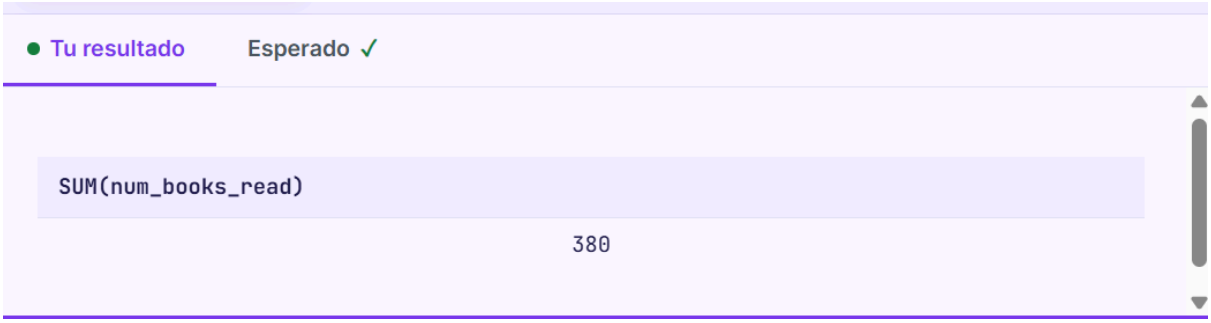
- **04. Consulta Ejecutada:** SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta devuelve un único valor numérico que representa estrictamente la cantidad de animales cuya especie registrada es un perro, omitiendo al resto de las especies.
- **07. Aprendizaje:** Comprendí cómo fusionar funciones agregadas con filtros condicionales para extraer métricas segmentadas y precisas en lugar de realizar un conteo global de toda la tabla.
- **08. Aplicación en el Mundo Real:** Es fundamental en la generación de reportes y paneles analíticos (*dashboards*) para obtener indicadores específicos, como contar cuántos usuarios activos tienen un rol determinado, cuántos productos pertenecen a una categoría o cuántas transacciones cumplen con un estado de pago.

Ficha 14: Sumatoria de valores numéricos con `SUM()`

- **01. Número y Nombre:** Ejercicio 14 - Suma de valores acumulados con la función `SUM()` (`SELECT SUM(num_books_read)`)
- **02. Objetivo del Ejercicio:** Calcular el número total acumulado de libros leídos por todos los integrantes de la familia en la tabla `family_members`.
- **03. Concepto SQL:** Función de agregación `SUM()` y sentencia `SELECT`.
- **04. Consulta Ejecutada:** SELECT SUM(num_books_read) FROM family_members;
- **05. Resultado (Captura de pantalla):**

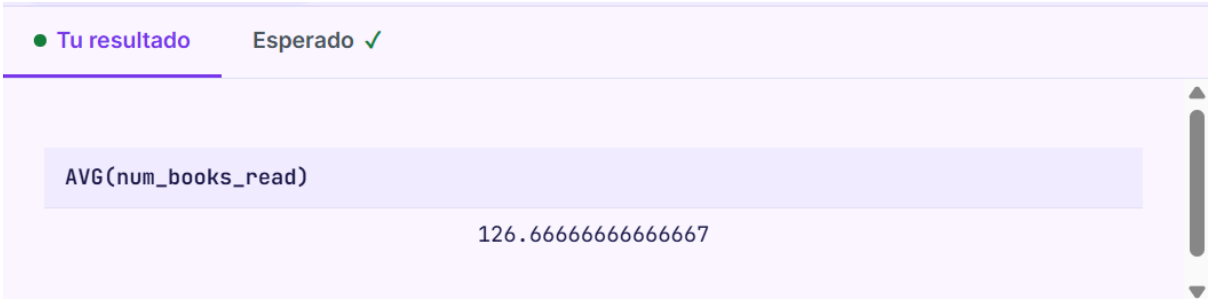


- **06. Interpretación:** La consulta devuelve un único valor numérico que representa la sumatoria total de todos los datos contenidos en la columna `num_books_read` para la totalidad de las filas de la tabla.

- **07. Aprendizaje:** Comprendí cómo utilizar la función `SUM()` para procesar y consolidar datos numéricos a nivel global, obteniendo métricas agregadas esenciales sin necesidad de sumar manualmente los registros.
- **08. Aplicación en el Mundo Real:** Es una herramienta fundamental en sistemas financieros, de comercio electrónico y gestión empresarial para calcular totales de gran volumen (por ejemplo, sumar el monto total de las ventas diarias, el costo total de un inventario o la cantidad global de horas invertidas en un proyecto).

Ficha 15: Cálculo del promedio con la función `AVG()`

- **01. Número y Nombre:** Ejercicio 15 - Cálculo del promedio de valores numéricos con la función `AVG()` (`SELECT AVG(num_books_read)`)
- **02. Objetivo del Ejercicio:** Obtener la media aritmética o promedio del número de libros leídos por los integrantes de la familia en la tabla `family_members`.
- **03. Concepto SQL:** Función de agregación `AVG()` y sentencia `SELECT`.
- **04. Consulta Ejecutada:** SELECT AVG(num_books_read) FROM family_members;
- **05. Resultado (Captura de pantalla):**

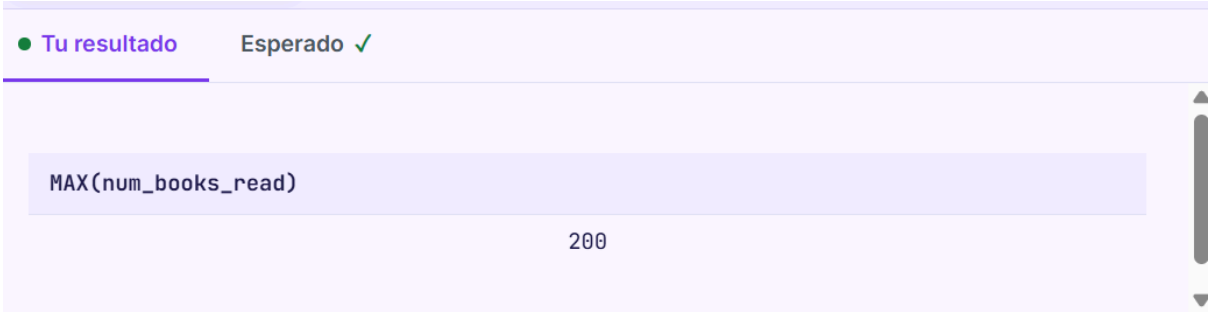


- **06. Interpretación:** La consulta devuelve un valor numérico (generalmente decimal) que representa el promedio aritmético de todos los datos contenidos en la columna `num_books_read` en relación con el total de filas de la tabla.
- **07. Aprendizaje:** Comprendí cómo utilizar la función `AVG()` para procesar rápidamente métricas de tendencia central sobre datos numéricos, evitando la necesidad de extraer y calcular los datos de forma manual.
- **08. Aplicación en el Mundo Real:** Es una herramienta fundamental en análisis de datos, sistemas financieros y paneles de control (*dashboards*) para calcular indicadores clave como el precio promedio de productos en un e-commerce, el tiempo promedio de resolución de incidencias o la calificación promedio de usuarios.

Ficha 16: Obtención del valor máximo con la función `MAX()`

- **01. Número y Nombre:** Ejercicio 16 - Obtención del valor máximo con la función `MAX()` (`SELECT MAX(num_books_read)`)
- **02. Objetivo del Ejercicio:** Encontrar el valor más alto de la columna `num_books_read` entre todos los integrantes de la familia en la tabla `family_members`.
- **03. Concepto SQL:**Función de agregación `MAX()` y sentencia `SELECT`.
- **04. Consulta Ejecutada:** SELECT MAX(num_books_read) FROM family_members;

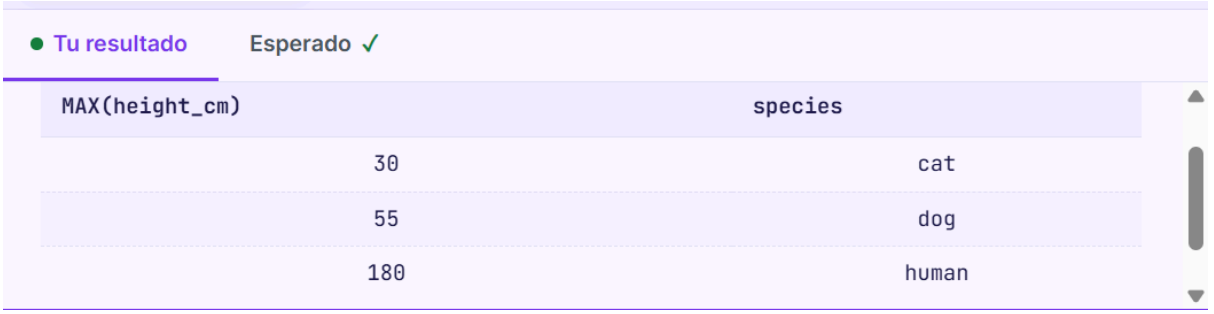
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta analiza la columna especificada y devuelve un único valor numérico que corresponde estrictamente al registro más alto de libros leídos dentro de todo el conjunto de datos de la familia.
- **07. Aprendizaje:** Comprendí cómo utilizar la función `MAX()` para aislar de forma directa el límite superior o valor máximo de una columna numérica, optimizando la obtención de extremos sin necesidad de aplicar un ordenamiento completo (`ORDER BY`) ni restricciones de filas (`LIMIT`).
- **08. Aplicación en el Mundo Real:** Es una herramienta fundamental en reportes de rendimiento, paneles de control y auditorías para identificar picos, récords o límites superiores (por ejemplo, conocer la venta de mayor monto, el salario más alto de una organización o la temperatura máxima registrada en un sensor).

Ficha 17: Agrupación y resumen de datos con `GROUP BY`

- **01. Número y Nombre:** Ejercicio 17 - Agrupación de registros y cálculo de máximos por categoría (`GROUP BY species`).
- **02. Objetivo del Ejercicio:** Obtener la altura máxima (*height_cm*) correspondiente a cada especie distinta registrada en la tabla `friends_of_pickles`.
- **03. Concepto SQL:** Cláusula `GROUP BY`, función de agregación `MAX()` y selección de columnas agrupadas.
- **04. Consulta Ejecutada:** SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;
- **05. Resultado (Captura de pantalla):**



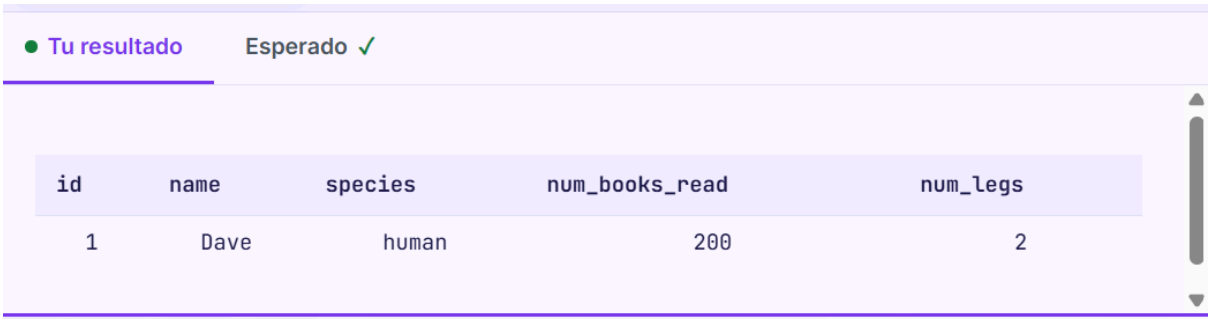
- **06. Interpretación:** La consulta divide la tabla en subgrupos basados en los valores únicos de la columna `species` y calcula de forma independiente el valor más alto de

la columna `height_cm` para cada grupo, devolviendo una fila por cada especie con su respectivo valor máximo.

- **07. Aprendizaje:** Comprendí el funcionamiento de la cláusula `GROUP BY` para categorizar grandes volúmenes de datos y aplicar funciones de agregación por subgrupos, lo que transforma un único valor global en un reporte segmentado y estructurado.
- **08. Aplicación en el Mundo Real:** Es una técnica indispensable en bases de datos relacionales, analítica web y sistemas de gestión para generar reportes por categorías (por ejemplo, calcular el precio máximo o promedio de productos divididos por cada categoría, o el total de ventas agrupado por sucursal).

Ficha 18: Consultas anidadas (Subconsultas) con `WHERE`

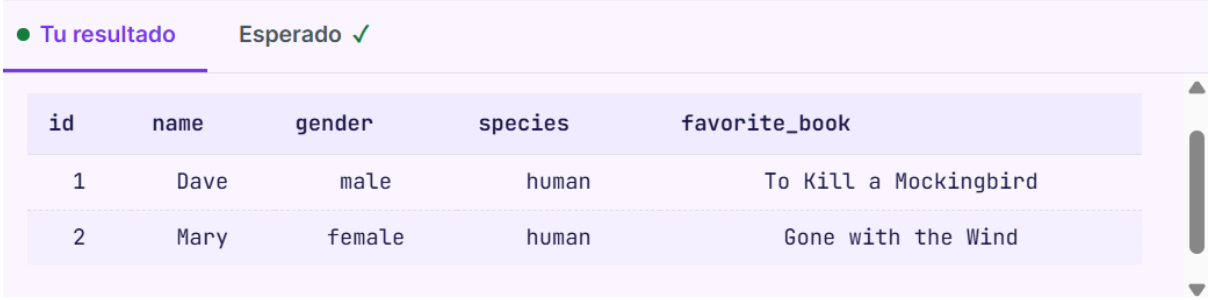
- **01. Número y Nombre:** Ejercicio 18 - Filtrado mediante subconsulta anidada
- **02. Objetivo del Ejercicio:** Encontrar y devolver los registros completos de los integrantes de la familia que han leído la mayor cantidad de libros, utilizando una consulta interna para calcular el valor máximo.
- **03. Concepto SQL:** Consultas anidadas (subconsultas), cláusula `WHERE` y función de agregación `MAX()`.
- **04. Consulta Ejecutada:** SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La subconsulta entre paréntesis (`SELECT MAX(...)`) se ejecuta primero para determinar de forma dinámica el número máximo de libros leídos. Una vez obtenido ese valor, la consulta principal lo utiliza como parámetro de comparación en la cláusula `WHERE` para filtrar y mostrar exclusivamente al miembro o miembros que empatan con ese récord.
- **07. Aprendizaje:** Comprendí el poder de las subconsultas para resolver problemas dinámicos donde el criterio de búsqueda no es un valor fijo, sino un resultado calculado analíticamente sobre los propios datos de la base de datos.
- **08. Aplicación en el Mundo Real:** Es una herramienta avanzada fundamental en la optimización de reportes y sistemas empresariales para aislar registros extremos sin necesidad de conocer de antemano el valor límite (por ejemplo, buscar empleados que ganan más que el promedio de la empresa, o clientes que realizaron la compra de mayor importe).

Ficha 19: Filtrado de datos nulos con `IS NOT NULL`

- **01. Número y Nombre:** Ejercicio 19 - Verificación de valores no nulos
- **02. Objetivo del Ejercicio:** Recuperar todos los registros de la tabla `family_members` donde la columna `favorite_book` contenga un valor asignado y no esté vacía (`NULL`).
- **03. Concepto SQL:** Cláusula `WHERE` y el operador condicional de nulidad `IS NOT NULL`.
- **04. Consulta Ejecutada:** SELECT * FROM family_members WHERE favorite_book IS NOT NULL;
- **05. Resultado (Captura de pantalla):**

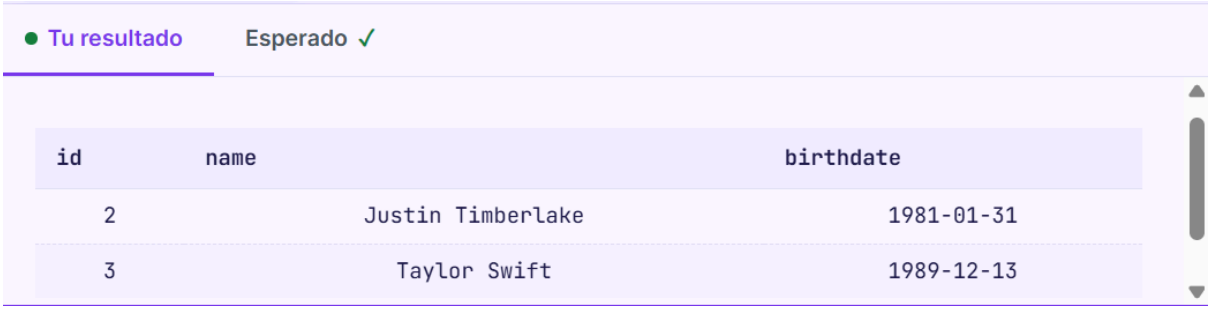


- **06. Interpretación:** La consulta examina la columna `favorite_book` en la tabla `family_members` y filtra el resultado para devolver exclusivamente aquellas filas que poseen información registrada, omitiendo de forma automática los registros donde el campo se encuentre ausente (`NULL`).
- **07. Aprendizaje:** Comprendí que los valores `NULL` representan la ausencia total de datos y que no pueden evaluarse mediante operadores de comparación tradicionales (como = o `!=`), siendo estrictamente necesario emplear `IS NULL` o `IS NOT NULL` para consultar su estado.
- **08. Aplicación en el Mundo Real:** Comprendí que los valores `NULL` representan la ausencia total de datos y que no pueden evaluarse mediante operadores de comparación tradicionales (como = o `!=`), siendo estrictamente necesario emplear `IS NULL` o `IS NOT NULL` para consultar su estado.

Ficha 20: Filtrado de registros cronológicos mediante fechas (>)

- **01. Número y Nombre:** Ejercicio 20 - Filtrado de celebridades nacidas después de una fecha específica
- **02. Objetivo del Ejercicio:** Obtener un listado completo de los registros de la tabla `celebs_born` correspondientes a celebridades cuya fecha de nacimiento sea posterior al 1 de septiembre de 1980.
- **03. Concepto SQL:** Comparación de campos de fecha utilizando el formato estándar `YYYY-MM-DD` en combinación con operadores relacionales (>) y la cláusula `WHERE`.
- **04. Consulta Ejecutada:** SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';

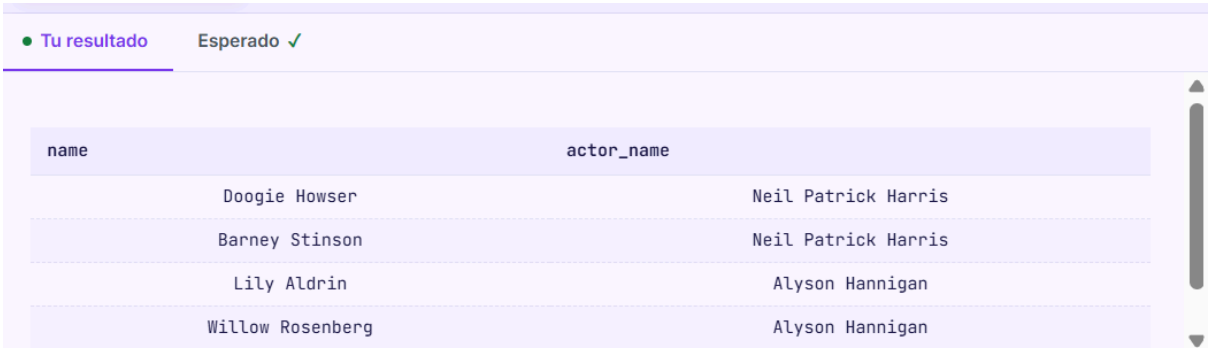
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta analiza la columna `birthdate` y descarta todos los registros anteriores o iguales a la fecha de corte, devolviendo únicamente aquellas filas donde la fecha de nacimiento es cronológicamente posterior al límite establecido.
- **07. Aprendizaje:** Comprendí que las bases de datos permiten procesar y comparar cadenas de texto estructuradas en formato de fecha estándar de manera cronológica, facilitando el uso de operadores matemáticos tradicionales (`<`, `>`, `<=`, `>=`) para aislar periodos de tiempo específicos.
- **08. Aplicación en el Mundo Real:** Es un requisito fundamental en sistemas de gestión, aplicaciones financieras y plataformas web para filtrar transacciones, registros de actividad, auditorías o perfiles de usuario según rangos de fechas (por ejemplo, buscar usuarios nacidos después de cierto año o transacciones realizadas tras el cierre de un periodo contable).

Ficha 21: Unión de tablas relacionadas con `INNER JOIN`

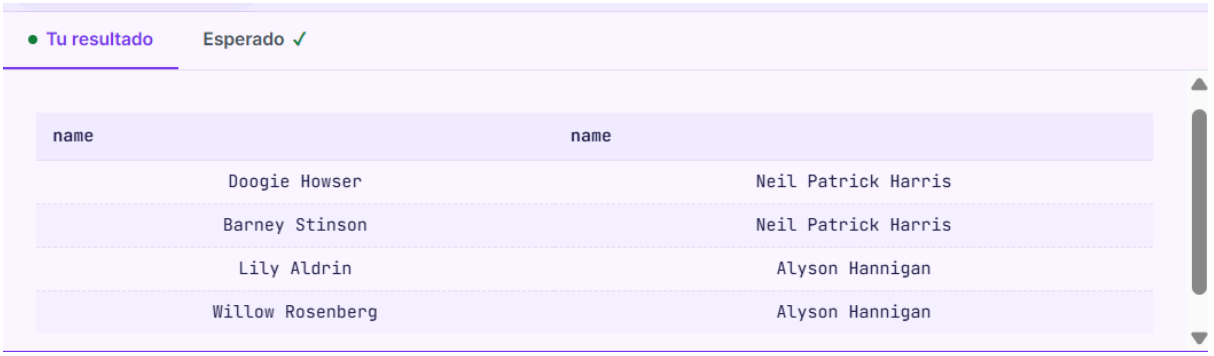
- **01. Número y Nombre:** Ejercicio 21 - Relación de personajes y actores mediante `INNER JOIN`
- **02. Objetivo del Ejercicio:** Emparejar y mostrar el nombre de cada personaje junto con el nombre del actor que lo interpreta, combinando información distribuida en dos tablas distintas (`character` y `character_actor`).
- **03. Concepto SQL:** Cláusula `INNER JOIN`, condición de enlace con `ON` y notación cualificada de columnas (`tabla.columna`).
- **04. Consulta Ejecutada:** SELECT character.name, character_actor.actor_name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id;
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta vincula la tabla `character` con la tabla `character_actor` haciendo coincidir el identificador único del personaje (`id`) con la clave foránea correspondiente (`character_id`), devolviendo un listado integrado que muestra exclusivamente las filas que tienen correspondencia en ambas tablas.
- **07. Aprendizaje:** Comprendí la importancia de la normalización de bases de datos para evitar la duplicación de información y cómo utilizar `INNER JOIN` para reconstruir y relacionar datos divididos en múltiples tablas mediante claves de referencia.
- **08. Aplicación en el Mundo Real:** Es una operación fundamental en sistemas de gestión de bases de datos relacionales, backends y reportes complejos para unificar información relacionada (por ejemplo, asociar clientes con sus pedidos, productos con sus categorías, o usuarios con sus permisos y roles dentro de una aplicación web).

Ficha 22: Uniones múltiples de tablas con `INNER JOIN` encadenados

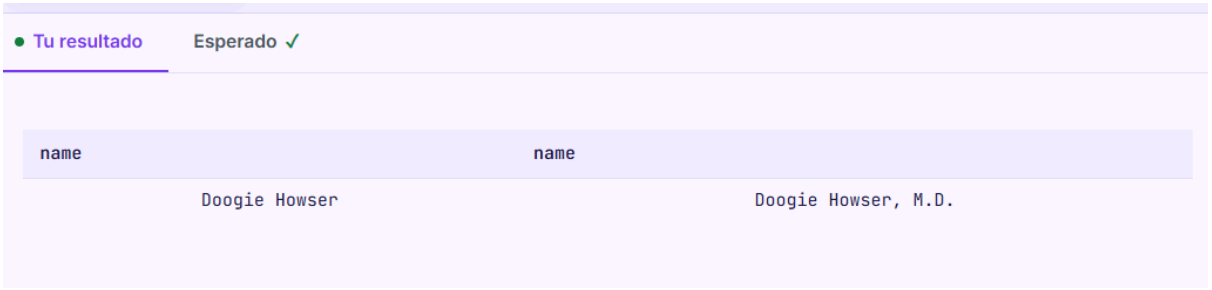
- **01. Número y Nombre:** Ejercicio 22- Relación de entidades mediante múltiples uniones
- **02. Objetivo del Ejercicio:** Emparejar el nombre de cada personaje con el nombre del actor que lo interpreta utilizando dos uniones consecutivas a través de una tabla intermedia de relación.
- **03. Concepto SQL:** Múltiples cláusulas `INNER JOIN ... ON`, tablas de relación (*junction tables*) y cualificación avanzada de columnas.
- **04. Consulta Ejecutada:** SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta parte de la tabla `character`, la conecta con la tabla intermedia `character_actor` mediante el identificador del personaje para acceder a las asociaciones y, posteriormente, realiza un segundo `INNER JOIN` con la tabla `actor` utilizando la clave del actor, logrando unificar la información de las tres tablas en un solo resultado coherente.
- **07. Aprendizaje:** Comprendí cómo escalar el uso de uniones para modelar y consultar relaciones complejas de muchos a muchos (*many-to-many*) mediante tablas intermedias, evitando la duplicación innecesaria de datos.
- **08. Aplicación en el Mundo Real:** Es una técnica indispensable en el diseño de bases de datos relacionales y consultas complejas de back-end para obtener reportes corporativos o de negocio que cruzan múltiples entidades (por ejemplo, asociar clientes con sus órdenes de compra y los detalles de los productos adquiridos).

  Ficha 23: Combinación de uniones múltiples con filtrado condicional (`INNER JOIN` y `WHERE`)

- **01. Número y Nombre:** Ejercicio 23 - Filtrado avanzado de registros en tablas unidas
- **02. Objetivo del Ejercicio:** Obtener un listado cruzado de personajes y programas de televisión excluyendo específicamente al personaje llamado "Willow Rosenberg" y al programa "How I Met Your Mother".
- **03. Concepto SQL:** Múltiples cláusulas `INNER JOIN`, filtrado condicional con `WHERE`, operador de desigualdad (`!=`) y operador lógico de conjunción (`AND`).
- **04. Consulta Ejecutada:** SELECT character.name, tv_show.name FROM character INNER JOIN character_tv_show ON character.id = character_tv_show.character_id INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id WHERE character.name != 'Willow Rosenberg' AND tv_show.name != 'How I Met Your Mother';
- **05. Resultado (Captura de pantalla):**



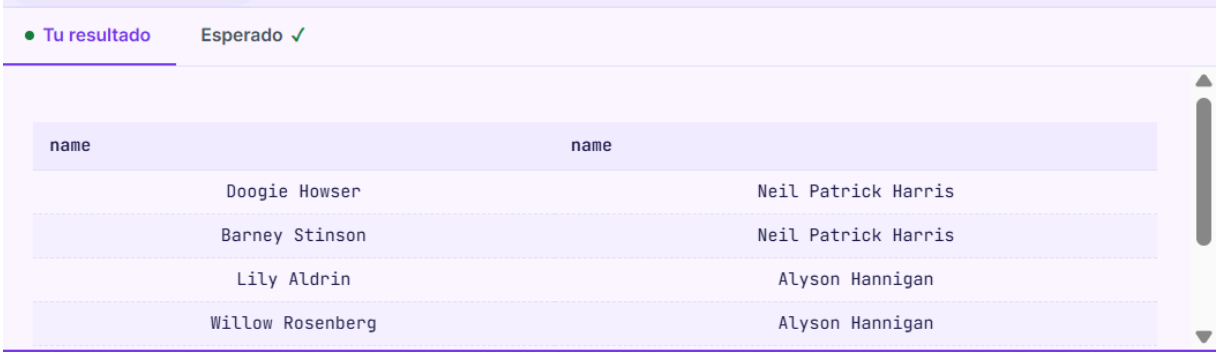
- **06. Interpretación:** La consulta unifica la información de las tres tablas (`character`, `character_tv_show` y `tv_show`) para asociar cada personaje con su respectivo programa de televisión y, posteriormente, descarta del conjunto de resultados final aquellas filas que coincidan con las exclusiones indicadas en la cláusula `WHERE`.

- **07. Aprendizaje:** Comprendí cómo integrar uniones relacionales complejas de múltiples tablas con filtros restrictivos condicionales, permitiendo refinar y acotar grandes volúmenes de datos cruzados según reglas específicas de exclusión.
- **08. Aplicación en el Mundo Real:** Es una técnica sumamente útil en la administración de bases de datos y desarrollo backend para generar reportes dinámicos que descartan registros obsoletos, bloqueados o no deseados de conjuntos de datos relacionales complejos (por ejemplo, listar pedidos activos excluyendo a ciertos clientes o categorías específicas).

Ficha 24: Preservación de registros huérfanos con `LEFT JOIN`

- **01. Número y Nombre:** Ejercicio 24 - Unión izquierda de personajes y actores
- **02. Objetivo del Ejercicio:** Obtener la lista completa de todos los personajes registrados, asegurando que se muestran incluso aquellos que no tengan un actor asignado en las tablas relacionadas.
- **03. Concepto SQL:** Cláusula `LEFT JOIN` y manejo de valores nulos (`NULL`) en relaciones condicionales.

- **04. Consulta Ejecutada:** SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;
- **05. Resultado (Captura de pantalla):**



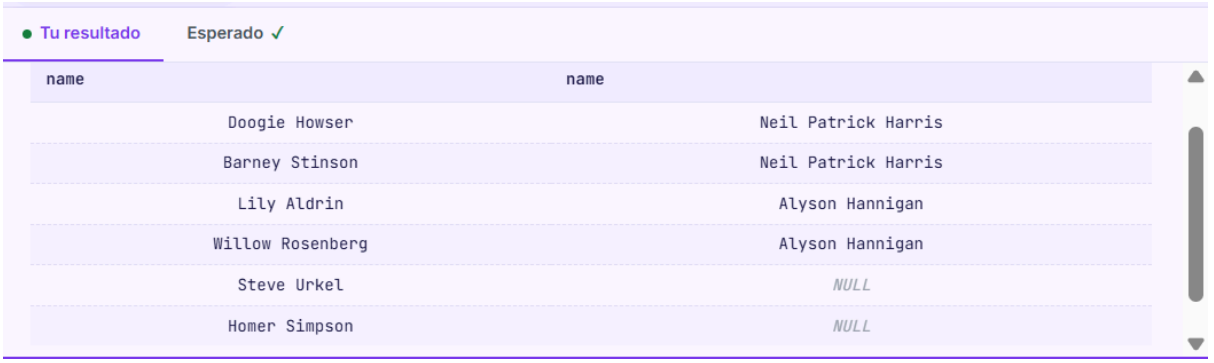
- **06. Interpretación:** La consulta parte de la tabla izquierda principal (`character`) y la conecta mediante `LEFT JOIN` a través de la tabla intermedia con la entidad `actor`. A diferencia de un `INNER JOIN`, esta instrucción garantiza que ningún personaje sea omitido; si un personaje no cuenta con un registro asociado en la tabla de actores, la consulta lo incluye de todos modos y devuelve `NULL` en el campo del actor.
- **07. Aprendizaje:** Comprendí la diferencia crítica entre `INNER JOIN` (que filtra estrictamente los registros que tienen correspondencia en ambos lados) y `LEFT JOIN` (que prioriza la tabla izquierda conservando todos sus registros sin importar si existen coincidencias o no).

- **08. Aplicación en el Mundo Real:** Es una técnica fundamental en análisis de bases de datos y desarrollo backend para generar reportes completos sin perder visibilidad sobre elementos sin dependencias (por ejemplo, listar todos los clientes de una plataforma, mostrando los detalles de sus compras recientes, pero sin excluir a los nuevos usuarios que aún no han realizado ninguna transacción).

Ficha 25: Optimización de consultas con alias de tabla (AS)

- **01. Número y Nombre:** Ejercicio 25 - Simplificación de consultas mediante alias de tabla.
- **02. Objetivo del Ejercicio:** Emparejar los nombres de los personajes con los actores que los interpretan utilizando uniones izquierdas, optimizando la legibilidad y longitud de la consulta a través de alias (*apodos*) de tabla asignados.
- **03. Concepto SQL:** Alias de tablas con la palabra clave AS, abreviación de referencias cualificadas y uniones múltiples (`LEFT JOIN`).
- **04. Consulta Ejecutada:** SELECT c.name, a.name FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;

- **05. Resultado (Captura de pantalla):**

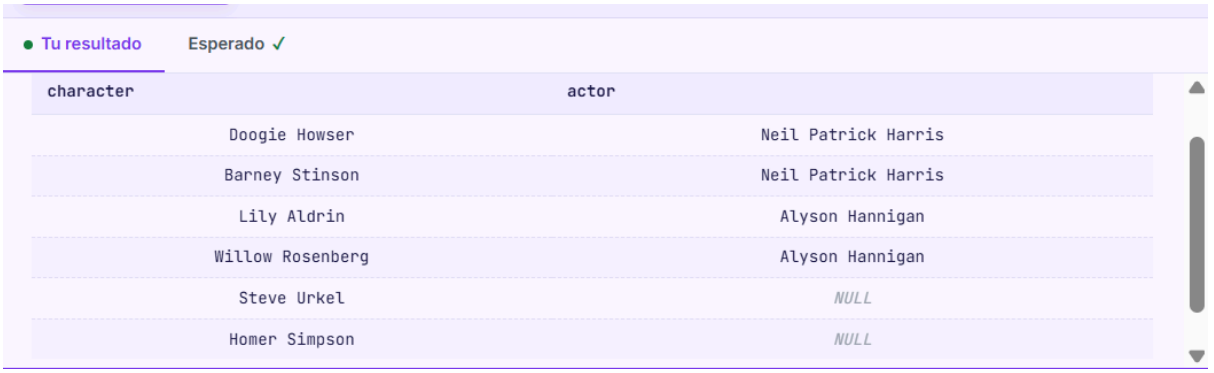


name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	NULL
Homer Simpson	NULL

- **06. Interpretación:** La instrucción vincula las tablas utilizando apodos cortos (c para `character`, ca para `character_actor` y a para `actor`), lo que simplifica de manera notable la estructura visual de la consulta tanto en la selección de columnas (SELECT) como en las condiciones de enlace (ON), preservando exactamente el mismo comportamiento lógico de una unión izquierda.
- **07. Aprendizaje:** Comprendí que los alias de tabla son una herramienta fundamental para escribir código SQL limpio, conciso y fácil de mantener, reduciendo la verbosidad excesiva al trabajar con múltiples relaciones entre bases de datos extensas.
- **08. Aplicación en el Mundo Real:** Es una práctica estándar y obligatoria en el desarrollo de software empresarial, consultas analíticas y vistas de bases de datos (*views*), facilitando la lectura de código complejo y previniendo errores de tipeo al interactuar con esquemas relacionales grandes.

Ficha 26: Claridad en los resultados con alias de columna (AS)

- **01. Número y Nombre:** Ejercicio 26 - Personalización de encabezados de salida mediante alias de columna.
- **02. Objetivo del Ejercicio:** Emparejar los nombres de los personajes con los actores que los interpretan utilizando uniones izquierdas (`LEFT JOIN`), renombrando explícitamente las columnas devueltas en el resultado como `character` y `actor` para evitar confusiones de nombres duplicados.
- **03. Concepto SQL:** Alias de columna con la palabra clave `AS`, uniones izquierdas y cualificación de tablas.
- **04. Consulta Ejecutada:** SELECT character.name AS character, actor.name AS actor FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;
- **05. Resultado (Captura de pantalla):**

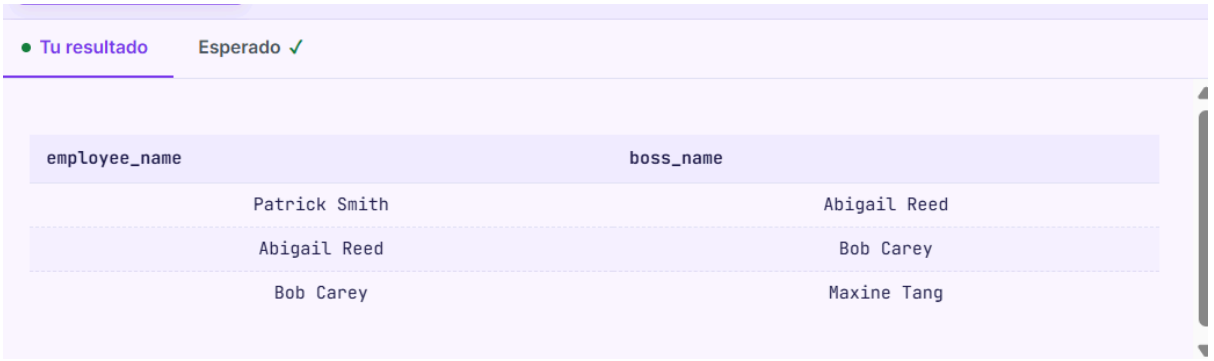


- **06. Interpretación:** La consulta recupera los datos cruzados de las tablas relacionadas y aplica la palabra clave AS directamente sobre los campos seleccionados (`name`), transformando los encabezados genéricos del resultado en nombres descriptivos y diferenciados (`character` y `actor`).
- **07. Aprendizaje:** Comprendí que los alias de columna permiten controlar con precisión la presentación de los datos devueltos por una consulta, resolviendo problemas de ambigüedad cuando múltiples tablas contienen columnas con nombres idénticos (como `name`).
- **08. Aplicación en el Mundo Real:** Es una práctica esencial en el desarrollo de software, diseño de reportes y construcción de consultas analíticas o vistas de bases de datos, ya que permite entregar información limpia, intuitiva y lista para ser consumida por usuarios finales, dashboards o interfaces de usuario.

Ficha 27: Autounión de tablas (`Self join`) para relaciones jerárquicas

- **01. Número y Nombre:** Ejercicio 27 - Relación de empleados y jefes mediante unión propia
- **02. Objetivo del Ejercicio:** Obtener un listado que relacione el nombre de cada empleado con el nombre de su respectivo jefe, aplicando una unión sobre una misma tabla (*self join*) con alias diferenciados.

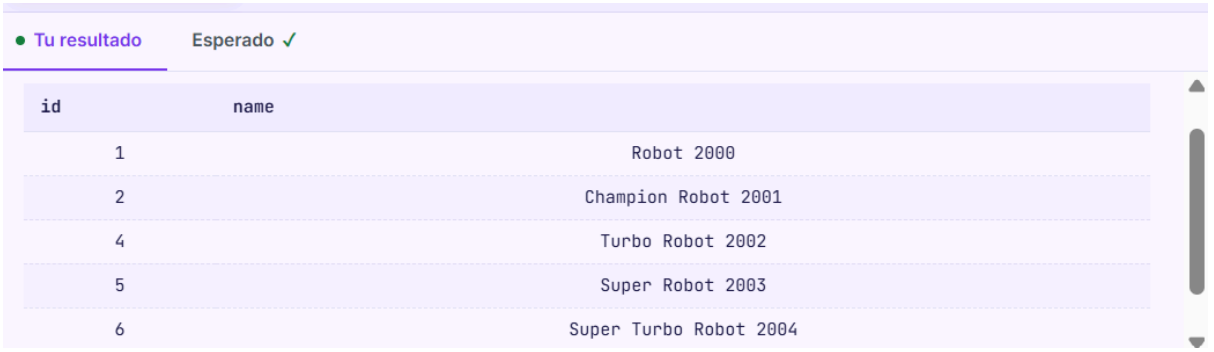
- **03. Concepto SQL:** Autounión (*Self join*), alias de tabla (`AS e1`, `AS e2`) y alias de columna (`AS employee_name`, `AS boss_name`).
- **04. Consulta Ejecutada:** SELECT e1.name AS employee_name, e2.name AS boss_name FROM employees AS e1 INNER JOIN employees AS e2 ON e1.boss_id = e2.id;
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta duplica virtualmente la tabla `employee` utilizando los alias e1 (representando a los empleados) y e2 (representando a los jefes), y las enlaza haciendo coincidir el identificador del jefe del empleado (`e1.boss_id`) con el identificador único del registro en la tabla de jefes (`e2.id`), renombrando los resultados para una lectura clara.
- **07. Aprendizaje:** Comprendí cómo consultar estructuras de datos jerárquicas o reflexivas donde una entidad se relaciona consigo misma, utilizando alias de tabla indispensables para evitar ambigüedades en esquemas de base de datos relacionales.
- **08. Aplicación en el Mundo Real:** Es una técnica fundamental en sistemas empresariales para modelar organigramas corporativos, estructuras de gestión de personal, árboles genealógicos o categorías y subcategorías anidadas en un catálogo de productos.

Ficha 28: Búsqueda de patrones de texto avanzados con `LIKE`, `%` y `_`

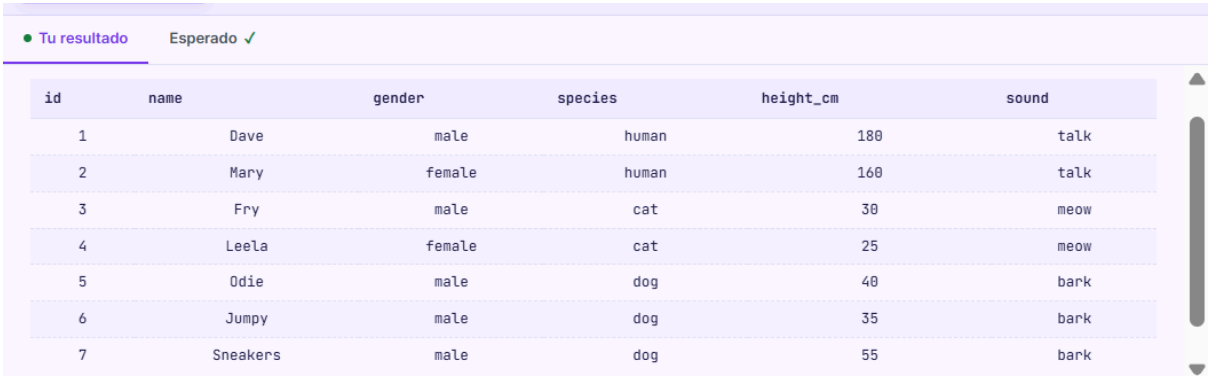
- **01. Número y Nombre:** Ejercicio 28 - Filtrado por patrones de texto con comodines.
- **02. Objetivo del Ejercicio:** Encontrar registros en la tabla `robots` cuyos nombres contengan la palabra "Robot" seguida de un año comprendido estrictamente entre 2000 y 2099.
- **03. Concepto SQL:** Operador `LIKE`, comodín de múltiples caracteres (`%`) y comodín de carácter individual (`_`).
- **04. Consulta Ejecutada:** SELECT * FROM robots WHERE name LIKE "%Robot 20__%";
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La consulta examina la columna `name` buscando el patrón especificado. El comodín `%` permite que haya cualquier texto antes o después, mientras que el bloque `20__` utiliza dos guiones bajos (`_`) para obligar a que los dos dígitos siguientes a "20" sean exactamente cualquier par de números, restringiendo el resultado únicamente a los años del siglo XXI (del 2000 al 2099) y descartando automáticamente otros siglos (como 2123).
- **07. Aprendizaje:** Comprendí cómo combinar los comodines `%` (para secuencias de caracteres arbitrarias) y `_` (para posiciones de caracteres fijos y específicos) con el operador `LIKE` para diseñar filtros de texto granulares y precisos.
- **08. Aplicación en el Mundo Real:** Es una técnica indispensable en motores de búsqueda, auditorías de bases de datos y sistemas de gestión para localizar códigos de serie, fechas parciales, formatos de correo electrónico, números de identificación o nombres que sigan una estructura estricta.

Ficha 29: Aplicación de lógica condicional al vuelo con `CASE`

- **01. Número y Nombre:** Ejercicio 29 - Creación de columnas condicionales con `CASE`.
- **02. Objetivo del Ejercicio:** Generar una columna calculada llamada `sound` que asigne dinámicamente una etiqueta de texto específica ("talk", "bark" o "meow") en función de la especie de cada registro en la tabla `friends_of_pickles`.
- **03. Concepto SQL:** Declaración condicional `CASE`, bloques `WHEN`, `THEN`, `ELSE`, `END` y alias de columna con `AS`.
- **04. Consulta Ejecutada:** SELECT *, CASE WHEN species = 'human' THEN 'talk' WHEN species = 'dog' THEN 'bark' WHEN species = 'cat' THEN 'meow' END AS sound FROM friends_of_pickles;
- **05. Resultado (Captura de pantalla):**



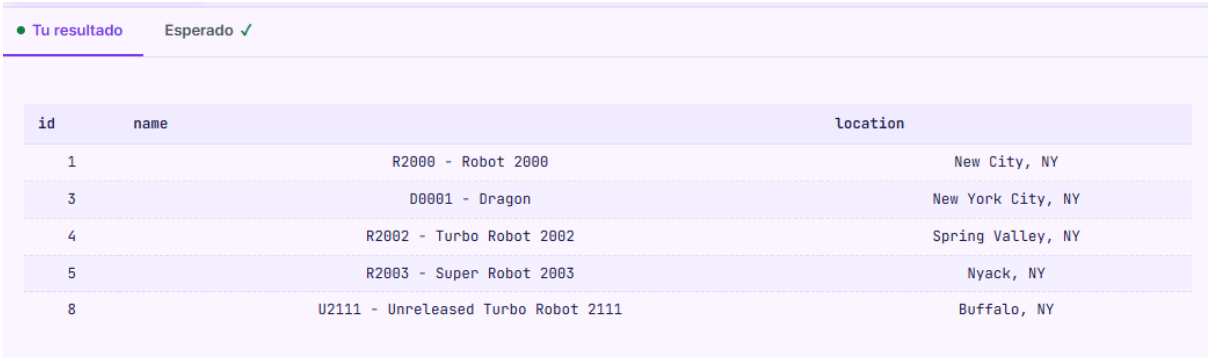
• Tu resultado Esperado ✓

id	name	gender	species	height_cm	sound
1	Dave	male	human	180	talk
2	Mary	female	human	160	talk
3	Fry	male	cat	30	meow
4	Leela	female	cat	25	meow
5	Odie	male	dog	40	bark
6	Jumpy	male	dog	35	bark
7	Sneakers	male	dog	55	bark

- **06. Interpretación:** La consulta examina de manera secuencial los valores de la columna `species` para cada fila. Cuando encuentra una coincidencia (por ejemplo, 'human'), devuelve el valor asociado ('talk') en una nueva columna virtual llamada `sound`, enriqueciendo el resultado sin alterar los datos originales almacenados en la tabla.
- **07. Aprendizaje:** Comprendí cómo implementar la estructura de control `CASE` para introducir lógica de tipo "si entonces" dentro de las consultas SQL, lo que permite transformar y categorizar datos al vuelo de acuerdo con reglas de negocio específicas.
- **08. Aplicación en el Mundo Real:** Es una herramienta analítica indispensable en el desarrollo de software y en la generación de reportes de inteligencia de negocios para clasificar rangos de datos, etiquetar estados de transacciones, calcular tramos de precios o adaptar formatos de información para los usuarios finales.

Ficha 30: Extracción y manipulación de texto con SUBSTR

- **01. Número y Nombre:** Ejercicio 30 - Filtrado por subcadena desde el final
- **02. Objetivo del Ejercicio:** Obtener todos los registros de la tabla `robots` correspondientes a aquellos ubicados en el estado de Nueva York ("NY"), extrayendo los caracteres finales de la columna de ubicación.
- **03. Concepto SQL:** Función SUBSTR, uso de índices negativos para conteo inverso desde el final de la cadena y cláusula `WHERE`.
- **04. Consulta Ejecutada:** SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';
- **05. Resultado (Captura de pantalla):**



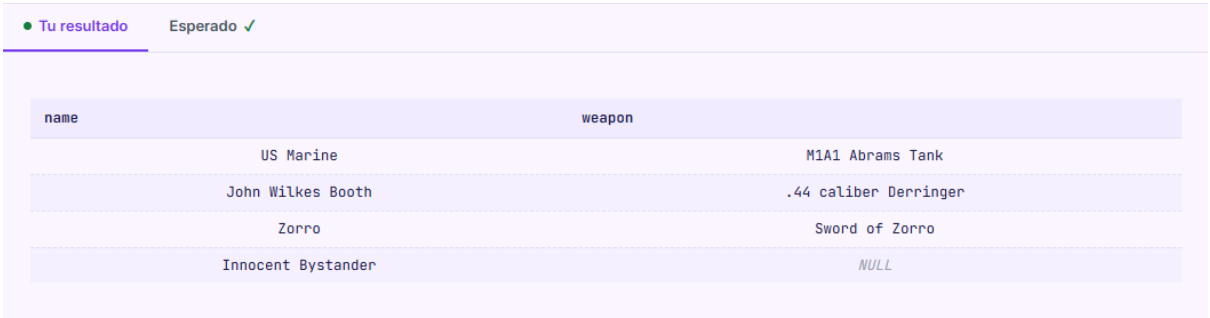
• Tu resultado Esperado ✓

id	name	location
1	R2000 - Robot 2000	New City, NY
3	D0001 - Dragon	New York City, NY
4	R2002 - Turbo Robot 2002	Spring Valley, NY
5	R2003 - Super Robot 2003	Nyack, NY
8	U2111 - Unreleased Turbo Robot 2111	Buffalo, NY

- **06. Interpretación:** La función SUBSTR(`location, -2`) examina la columna `location` contando desde el final de cada cadena de texto (el índice `-2` indica los últimos dos caracteres correspondientes a la abreviatura del estado). La cláusula WHERE compara ese fragmento extraído con el valor `'NY'`, filtrando y devolviendo únicamente las filas que coinciden con dicha condición.
- **07. Aprendizaje:** Comprendí cómo utilizar la función SUBSTR con índices negativos para aislar dinámicamente porciones específicas de cadenas de texto sin importar la longitud variable que posea el resto del contenido (como el nombre de la ciudad).
- **08. Aplicación en el Mundo Real:** Es una herramienta fundamental en la limpieza de datos (*data parsing*), análisis de registros y procesamiento de información en backends para extraer fragmentos estandarizados de campos de texto libre, tales como códigos postales, extensiones de archivos, dominios de correo o prefijos geográficos.

Ficha 31: Selección del primer valor no nulo con COALESCE

- **01. Número y Nombre:** Ejercicio 31 - Determinación de prioridad de armas mediante COALESCE.
- **02. Objetivo del Ejercicio:** Encontrar el arma disponible de mayor prioridad para cada combatiente evaluando múltiples columnas en orden jerárquico (dando prioridad al tanque, seguido del arma de fuego y por último la espada) y devolviendo el primer valor no nulo encontrado.
- **03. Concepto SQL:** Función COALESCE(`val1, val2, val3, ...`), manejo jerárquico de valores nulos y alias de columna (AS).
- **04. Consulta Ejecutada:** SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;
- **05. Resultado (Captura de pantalla):**



- **06. Interpretación:** La función COALESCE analiza las columnas especificadas de izquierda a derecha para cada fila. Si el combatiente tiene un valor en `tank` (no es NULL), lo devuelve de inmediato; si `tank` está vacío, pasa a evaluar `gun`, y si ambas son nulas, recurre a `sword`, asegurando que siempre se obtenga la mejor opción disponible sin errores de valores vacíos.
- **07. Aprendizaje:** Comprendí cómo la función COALESCE simplifica de forma elegante la lógica condicional para manejar valores alternativos o respaldos (*fallbacks*) en una sola línea, evitando el uso de múltiples estructuras CASE anidadas y complejas.

- **08. Aplicación en el Mundo Real:** Es una herramienta fundamental en el desarrollo de bases de datos, migraciones y reportes empresariales para gestionar datos incompletos o secundribles (por ejemplo, mostrar el teléfono principal de contacto de un usuario y, si no existe, recurrir automáticamente al teléfono secundario o al correo electrónico).

CONCLUSIONES DE LA FASE SQL

- **Temas más accesibles:**

  Operadores básicos y filtrado: Las consultas fundamentales empleando `SELECT`, `FROM`, `WHERE` y combinaciones simples con operadores relacionales (=, >, <) resultaron sumamente intuitivas para acotar información inicial.

  Ordenamiento y límite de datos: La implementación de `ORDER BY` (en orden ascendente o descendente) junto con `LIMIT` permitió controlar la presentación y acotación de resultados de manera clara y directa.

  Funciones de agregación simples: Operaciones como `COUNT()`, `SUM()`, `AVG()`, `MIN()` y `MAX()` se asimilaron con rapidez para obtener métricas y resúmenes cuantitativos básicos sobre los datos.

- **Temas más retadores:**

  Uniones múltiples y complejas (`JOIN`s): Entender la diferencia pragmática entre `INNER JOIN` y `LEFT JOIN` (especialmente al encadenar tres o más tablas con intermediarias de tipo muchos a muchos) requirió un mayor esfuerzo conceptual para visualizar la preservación o filtrado de datos nulos (*registros huérfanos*).

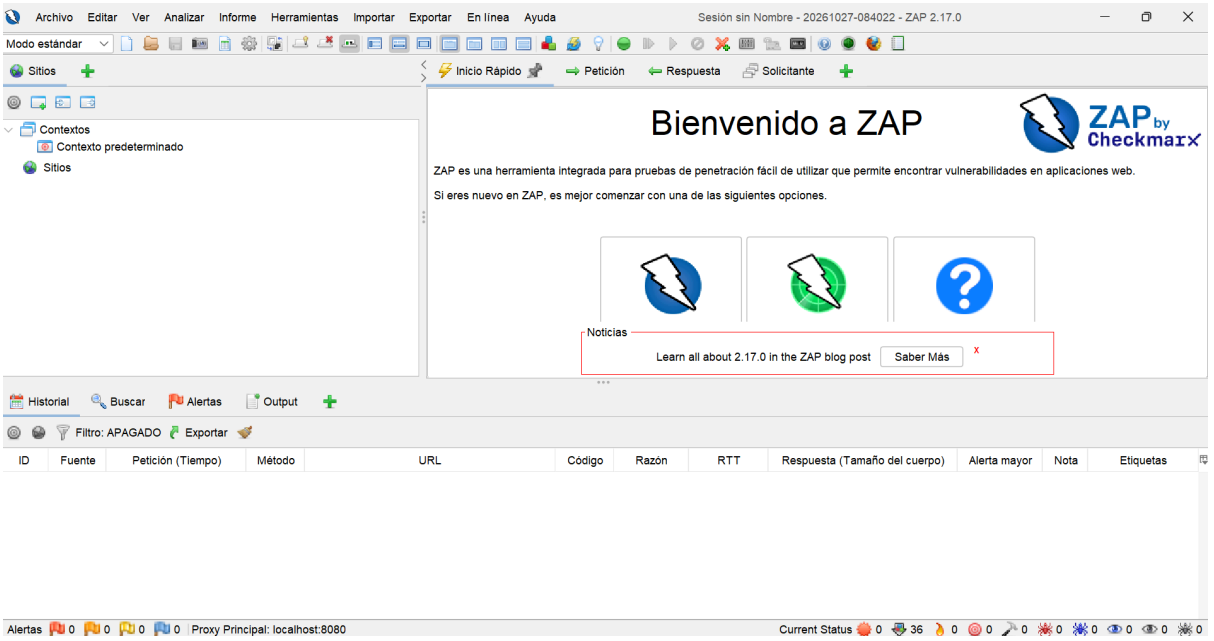
- **Autouniones (*Self Joins*):** Relacionar una tabla consigo misma para modelar jerarquías (como empleados y sus respectivos jefes) exigió abstraer el uso de alias de tabla (`AS e1`, `AS e2`) para evitar confusiones de contexto dentro de la misma entidad.
- **Lógica condicional y patrones de texto avanzadas:** La combinación de funciones de manipulaciones de subcadenas (SUBSTR con índices negativos), patrones con múltiples comodines en `LIKE` (`%` y `_`), e instrucciones dinámicas `CASE` demandó un análisis más riguroso para construir la lógica adecuada en un solo paso.
- **Conclusión General SQL:** El recorrido práctico a través de los 31 ejercicios de SQL permitió consolidar una base sólida en el lenguaje de consultas estructuradas, transitando desde la extracción de información básica hasta la manipulación analítica avanzada. Se comprendió que el diseño relacional de datos es clave para la integridad de la información y la prevención de redundancias. En el contexto de la ciberseguridad, dominar la sintaxis estándar y la lógica interna de la base de datos resulta imprescindible, ya que sirven para comprender cómo interactúan las aplicaciones web con el almacenamiento backend, facilitando el análisis de vulnerabilidades y la auditoría segura de repositorios de datos.

SECCIÓN 2: EVALUACIÓN DE SEGURIDAD WEB CON OWASP ZAP

Objetivo: https://ginandjuice.shop/

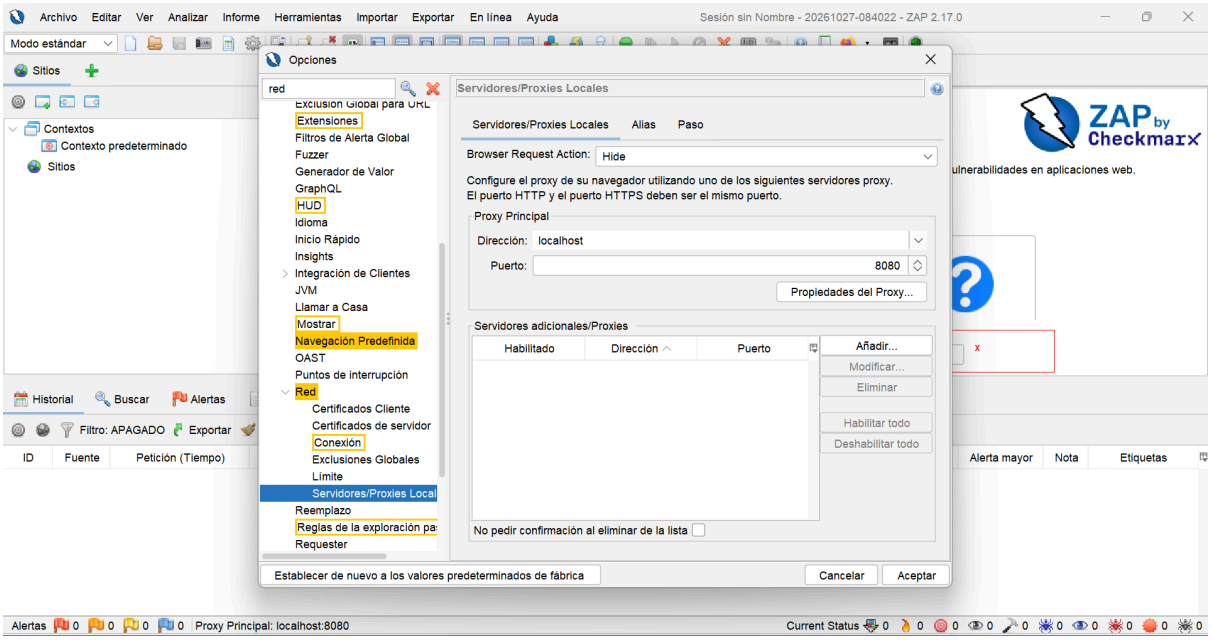
2.1 CONFIGURACIÓN DEL ENTORNO

1. **ZAP Abierto:**



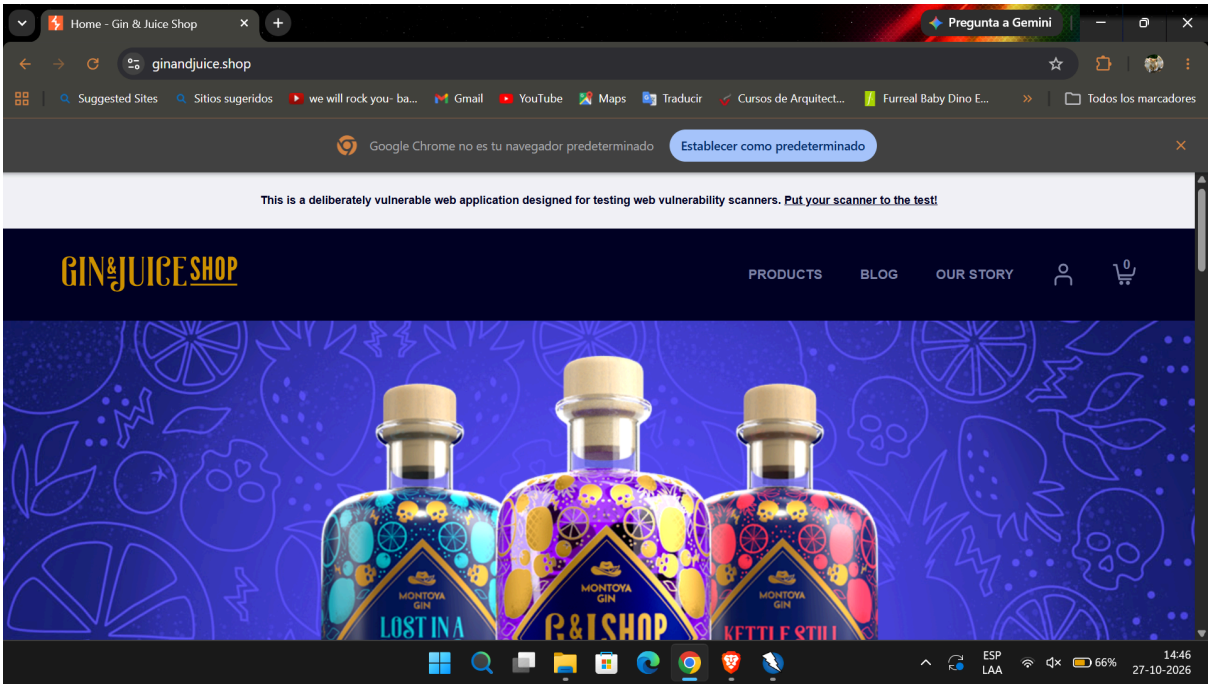
Captura interfaz principal de OWASP ZAP

2. **Navegador Configurado:**



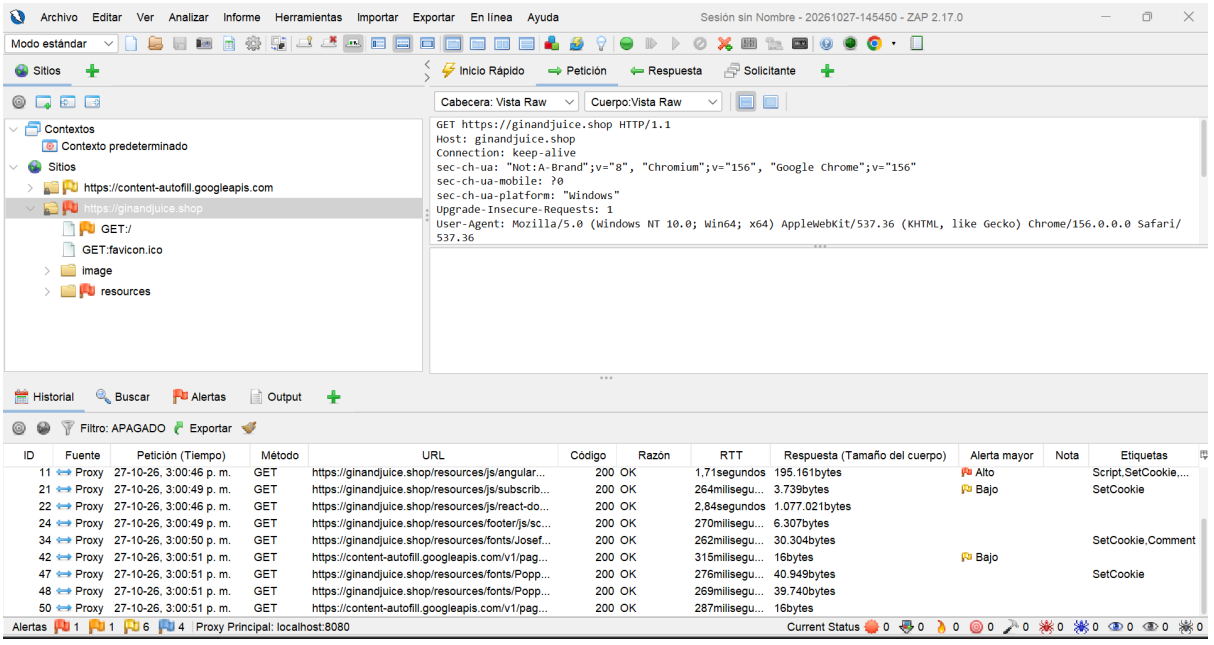
Captura ajustes de proxy del navegador apuntando a 127.0.0.1:8080

3. **Acceso al Sitio Web:**



Captura navegando en https://ginandjuice.shop

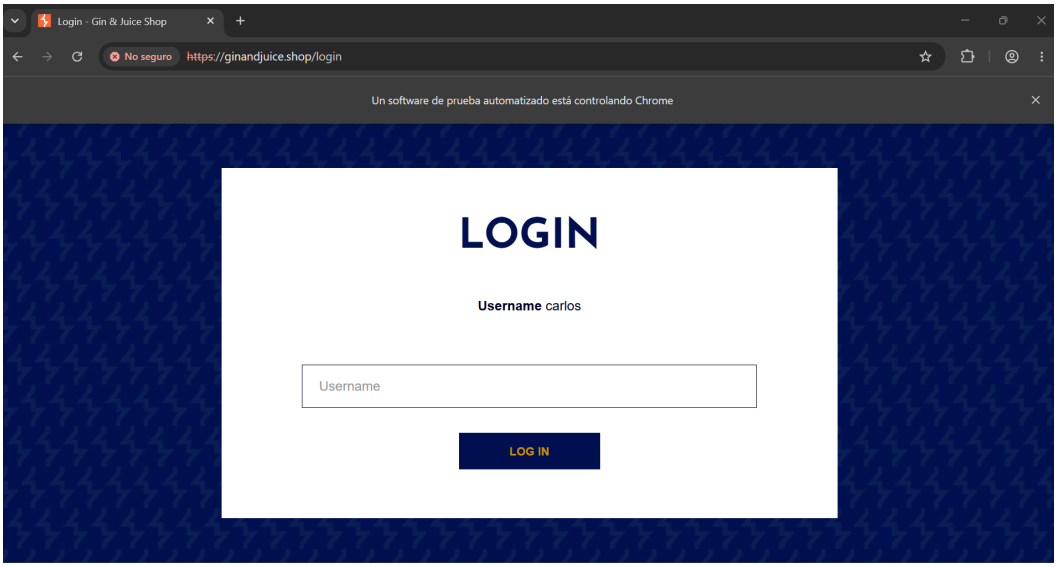
4. **Verificación de Tráfico en ZAP:**



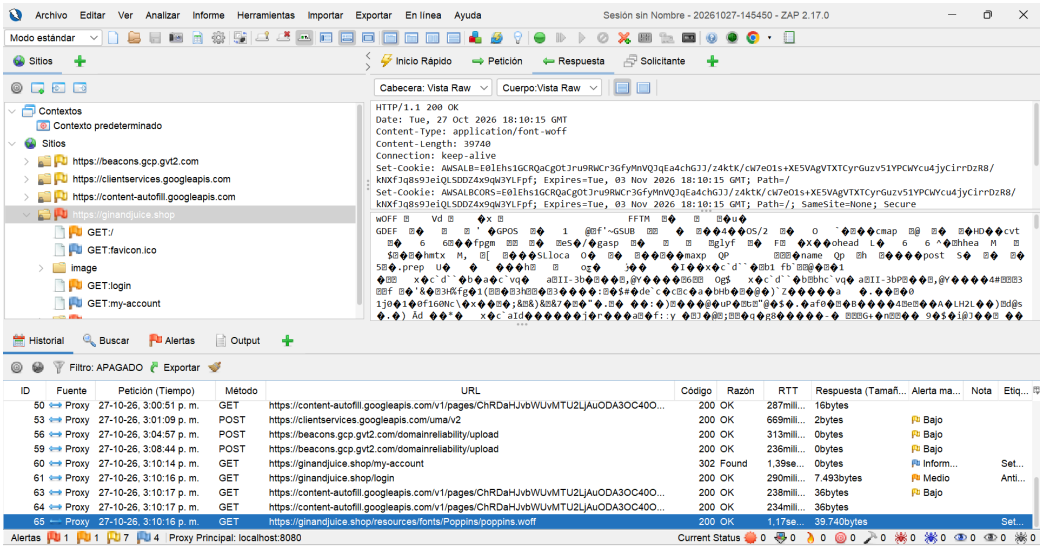
Captura de la pestaña History o Sites en ZAP mostrando peticiones hacia ginandjuice.shop

2.2 EXPLORACIÓN Y TRÁFICO DE LA APLICACIÓN

1. **Página de Inicio / Login:**



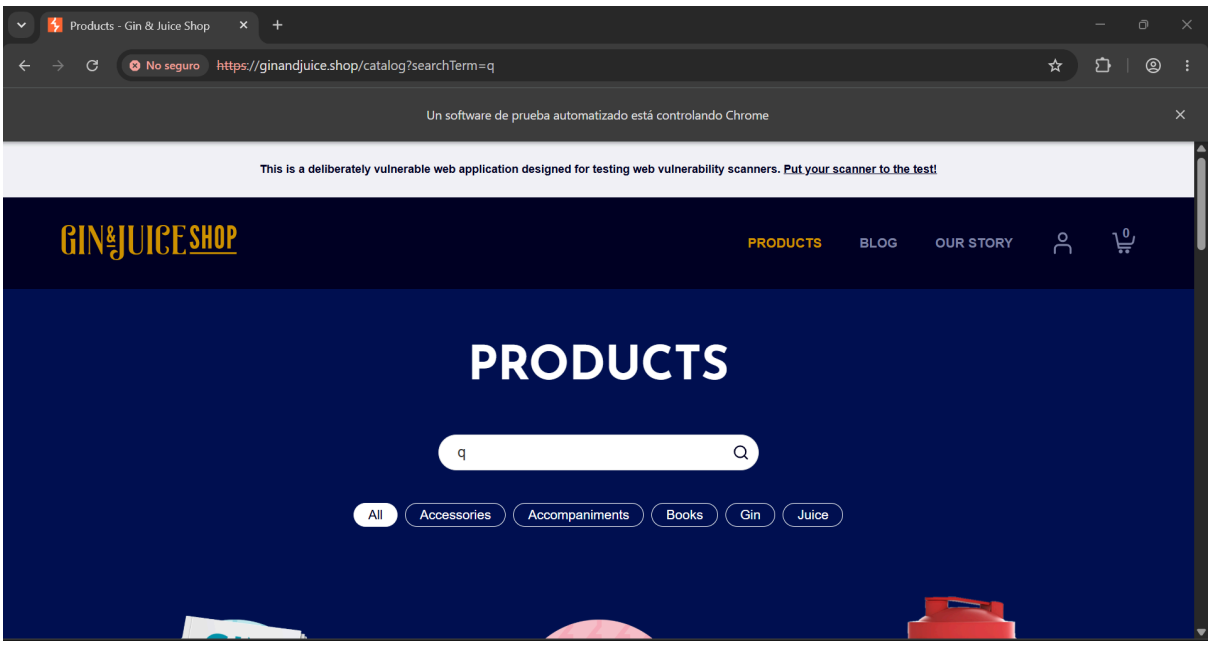
Captura página de Inicio / Login



Captura registro en ZAP del login

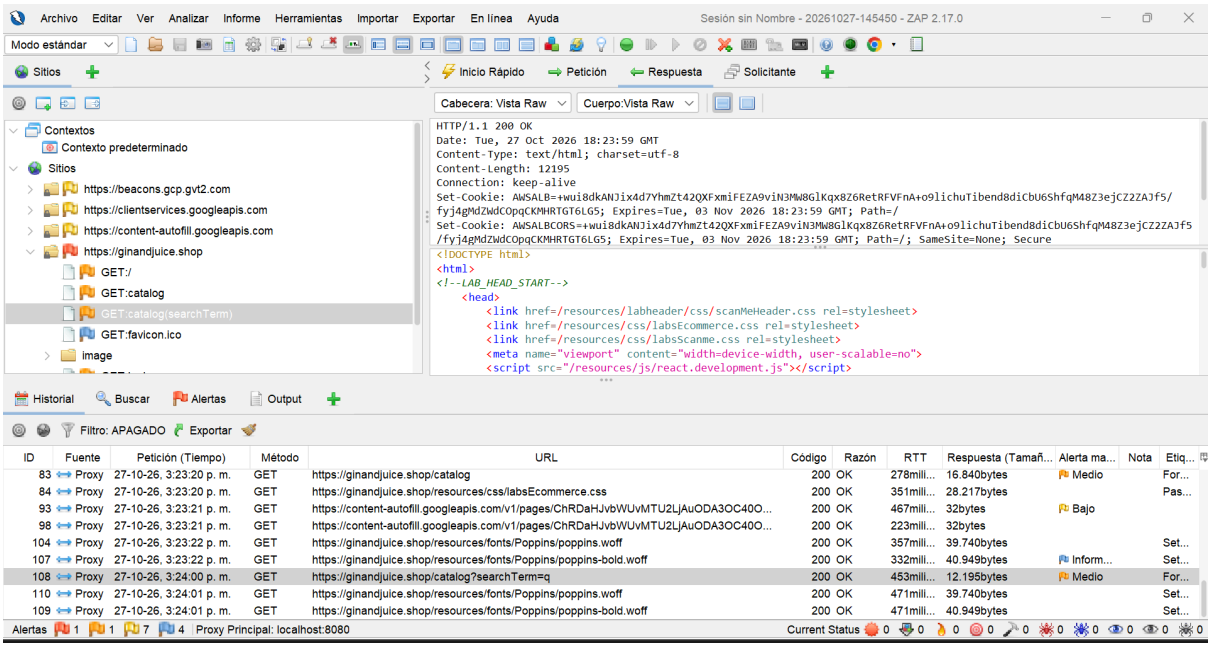
2. **Búsqueda de Productos:**

Descripción: Se evaluó la funcionalidad de búsqueda de artículos en la plataforma (interfaz web y endpoint `/rest/products/search`) ingresando parámetros de consulta para verificar el comportamiento del servidor ante entradas de usuario.



Evidencia 1 (Navegador Web)

Descripción: Intercepción y registro en el historial de ZAP de la petición HTTP GET hacia `/catalog?searchTerm=q` con respuesta exitosa `200 OK`.



Evidencia 2 (Registro de Tráfico en ZAP)

2.3 ANÁLISIS DE ALERTAS DETECTADAS

A continuación se resume la clasificación y cantidad de vulnerabilidades y configuraciones inseguras identificadas por OWASP ZAP sobre https://ginandjuice.shop.

#	Tipo de Alerta / Vulnerabilidad	Nivel de Riesgo	Descripción Breve
1	Librería JS Vulnerable (AngularJS v1.7.7)	**Alto**	Uso de componentes desactualizados con vulnerabilidades conocidas de XSS/Inyección
2	Cabecera CSP no configurada	Medio	Ausencia de Content Security Policy, aumentando el riesgo de ataques XSS e inclusión de scripts.
3	Servidor filtra versión (*Server Header*)	Medio	El servidor expone información de la infraestructura en las cabeceras HTTP.

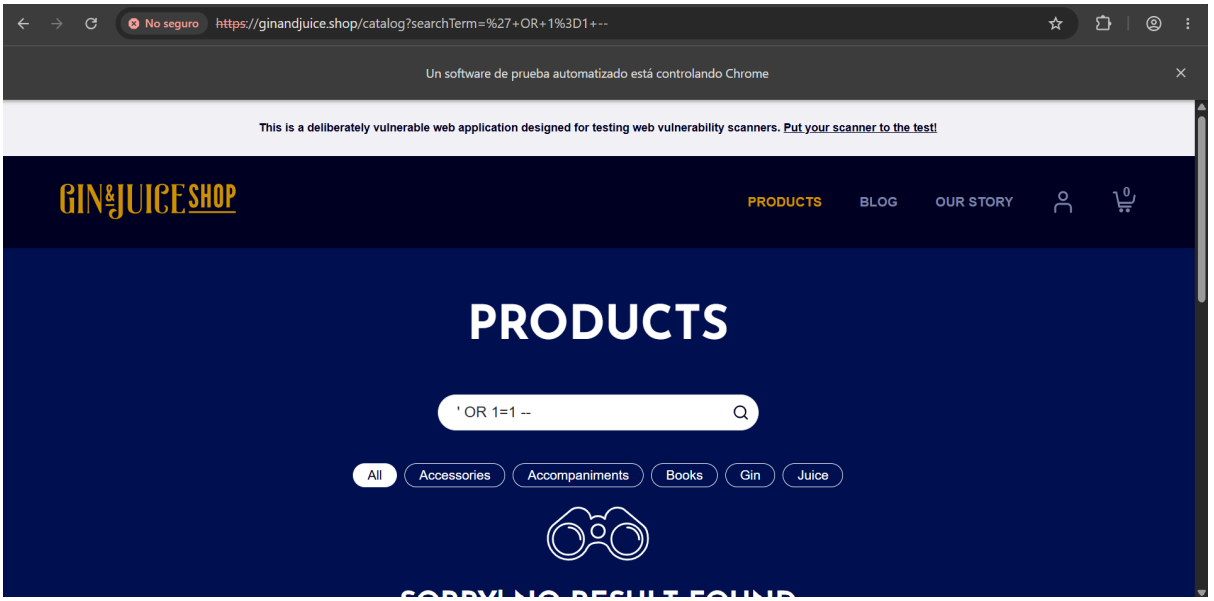
Evidencia :



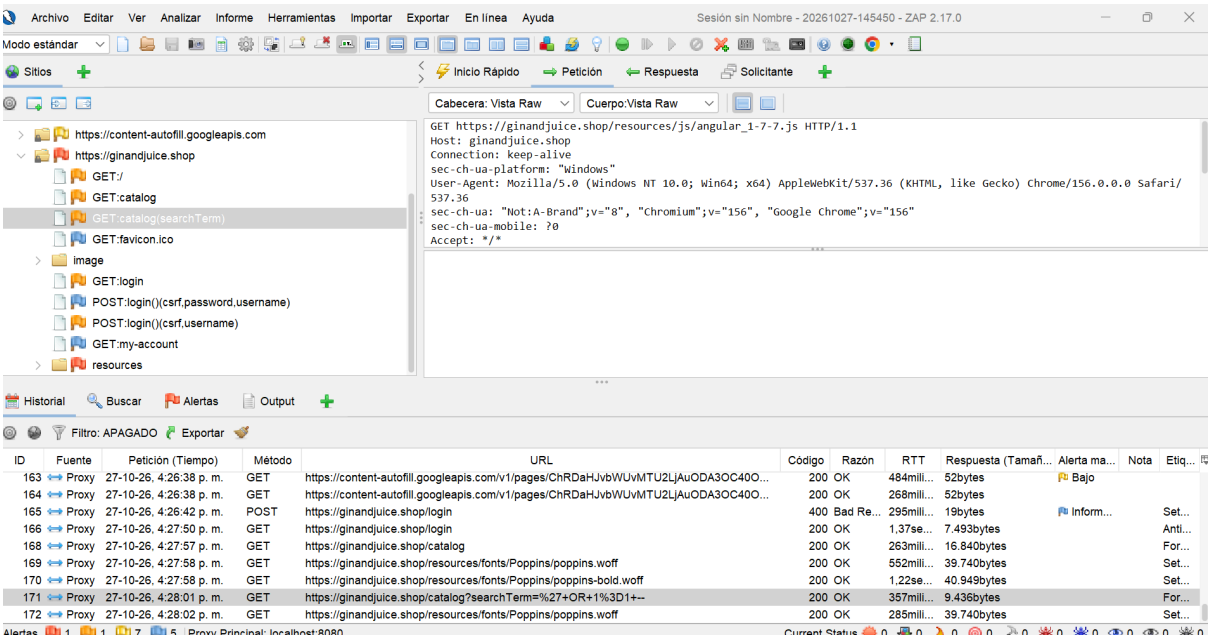
Captura clasificación de riesgos detectados en el escaneo de la aplicación.

2.4 FICHA TÉCNICA DE HALLAZGO PRINCIPAL

- **1. Nombre de la Alerta:** SQL Injection
- **2. Nivel de Riesgo:** Alto
- **3. URL o Componente:** https://ginandjuice.shop/test/products/search
- **4. Parámetro Afectado:** searchTerm
- **5. Evidencia (Captura / Traza HTTP):**



Captura Ingreso del payload (' OR 1=1 --) en el buscador de la tienda



Captura Traza HTTP interceptada por OWASP ZAP con el parámetro de búsqueda

6. Descripción Técnica: El parámetro "searchTerm" no sanitiza de forma adecuada las entradas del usuario ni utilizar consultas parametrizadas, permitiendo modificar la lógica de las sentencias SQL internas

7. Impacto Potencial: Acceso no autorizado a la base de datos, lectura o alteración de datos sensibles y evasión de mecanismos de autenticación o lógica de negocio.

8. Recomendación de Mitigación: Implementar consultas parametrizadas y aplicar la validación y sanitización estricta de entradas en el servidor.

CONCLUSIONES GENERALES DEL PROYECTO

1. **¿Qué aprendí?:**
   Adquisición de habilidades: Se fortalecieron competencias clave en la gestión de tráfico web y ciberseguridad defensiva y ofensiva mediante el uso de OWASP ZAP sobre entornos vulnerables controlados.
2. **¿Qué fue lo más retador?:**
   Comprender y relacionar el comportamiento de la aplicación web ante la introducción de caracteres especiales y payloads (como `' OR 1=1 --`) con su respectiva intercepción y codificación de parámetros en el historial y las peticiones HTTP interceptadas por el proxy de ZAP.
3. **¿Qué revisaré a continuación?** Profundización en análisis: Realizar pruebas de explotación dirigidas (por ejemplo, extracción controlada de bases de datos mediante herramientas como SQLmap) para medir el impacto real de la vulnerabilidad de SQL Injection descubierta.